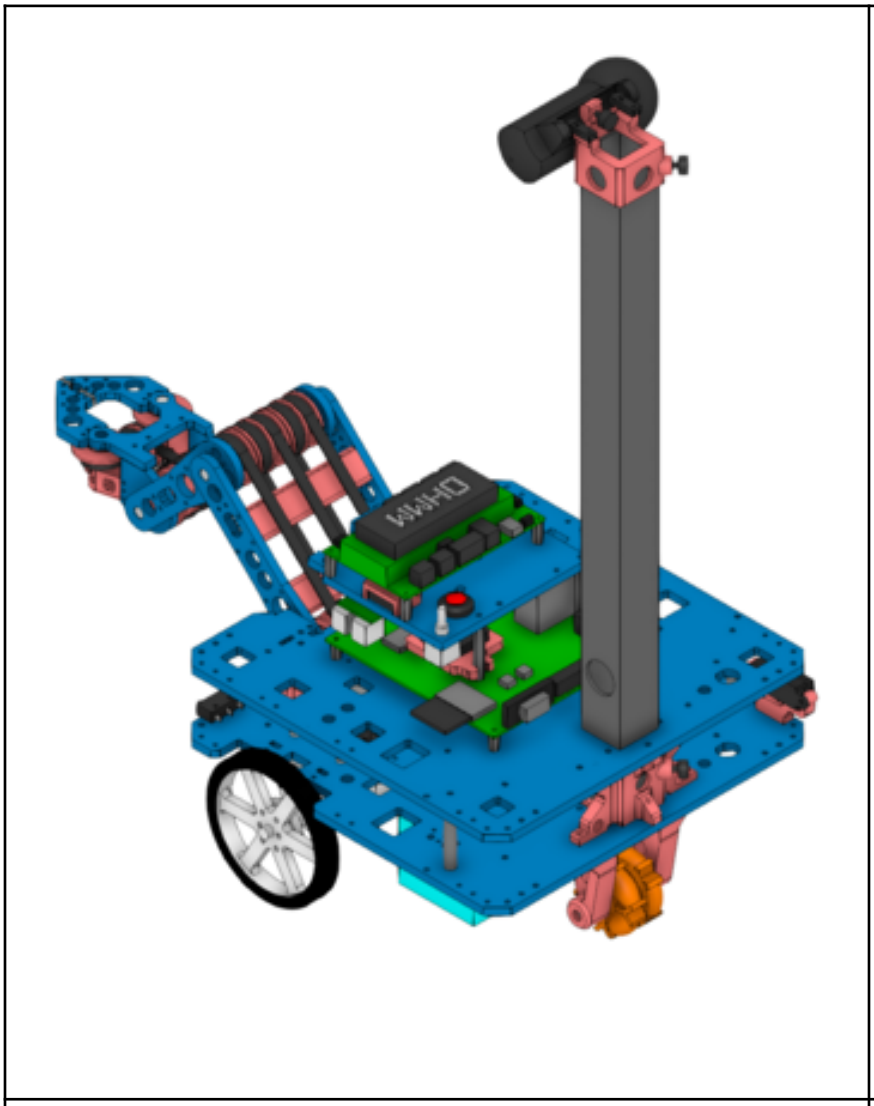
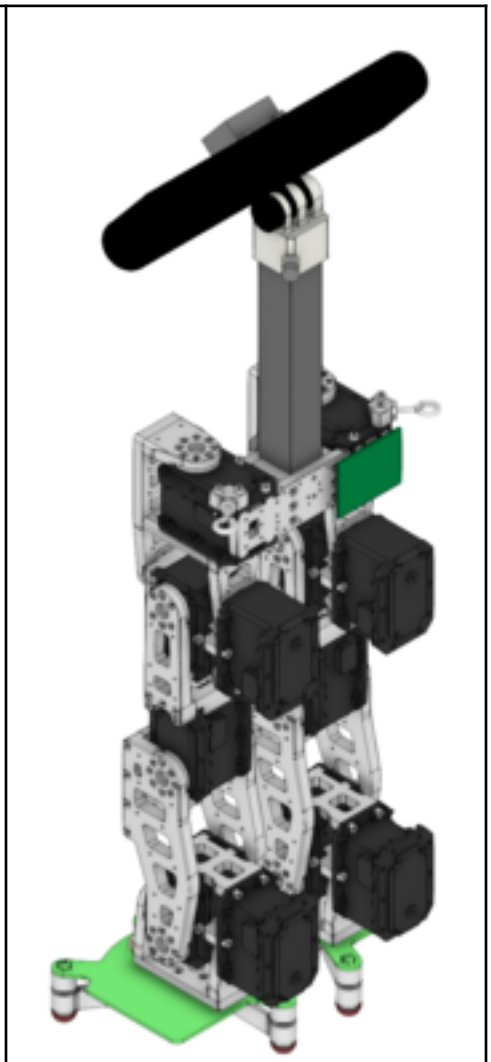
# cad-io-js

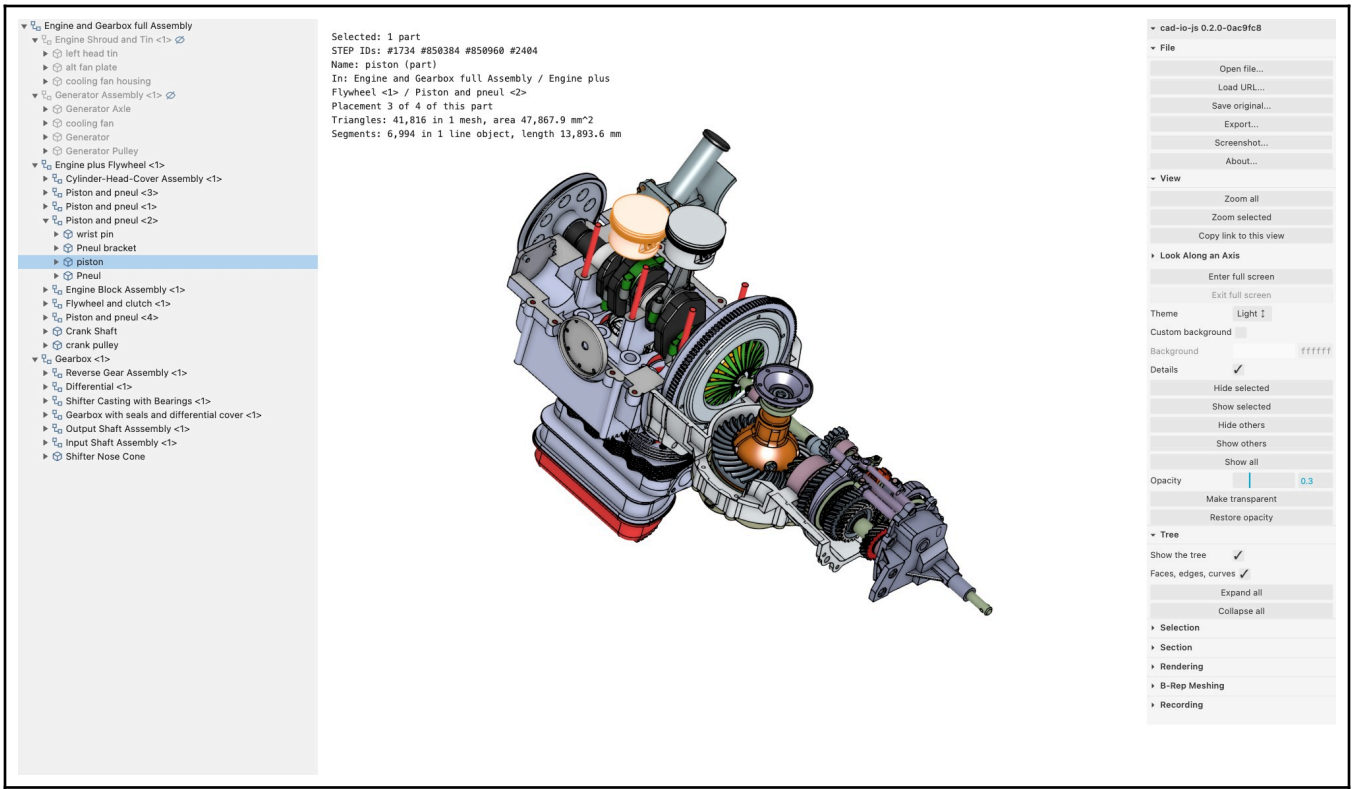
## Contents


1 Building From Source ..... 5
1.1 What You Need First ..... 5
1.2 Steps ..... 6
2 Installing ..... 7
3 Command-Line Tools ..... 7
3.1 `step2json`: STEP to JSON ..... 7
3.2 `stepconv`: STEP to three.js, glTF or 3MF ..... 8
4 Viewer ..... 11
4.1 Running It ..... 11
4.2 Using It ..... 11
4.3 Driving It ..... 15
4.4 Recording and Replaying ..... 16
4.5 What the Viewer Does with Files ..... 17
4.6 Building and Deploying ..... 17
4.7 Phones, Tablets and https ..... 18
4.8 Checks ..... 18
5 Architecture ..... 19
5.1 Package Dependencies ..... 19
5.2 From a STEP File to a Mesh File ..... 20
5.3 From glTF, three.js JSON or 3MF to the Scene ..... 21
5.4 step-parser ..... 22
5.5 Scene Model (cad-scene) ..... 23
5.6 Tessellation ..... 25
5.7 Exporters ..... 26
6 Package APIs ..... 26
6.1 `step-parser` (Part 21) ..... 26
6.2 `step-schemas` ..... 27
6.3 `lexical-json` ..... 28
6.4 `cad-scene` ..... 28
6.5 `step-import` ..... 30
6.6 `threejs-import` ..... 30
6.7 `gltf-import` ..... 31
6.8 `3mf-import` ..... 32
6.9 `tess-occt`, `tess-truck` ..... 32
6.10 `tess-workers` ..... 33
6.11 `tess-cache` ..... 34
6.12 `export-threejs`, `export-gltf`, `export-3mf` ..... 34
6.13 `cli` (`cad-io-cli`) ..... 36
7 Test Corpus ..... 36
8 Tests ..... 37
8.1 Corpus Checks ..... 37
9 Roadmap ..... 39
10 Backlog ..... 40

11 Contributing ..... 41
12 License ..... 41

Reading CAD files into memory, and converting them, in TypeScript for Node.js and browsers. Today that means STEP (ISO 10303-21 files of AP203, AP214 and AP242), and glTF/GLB, three.js JSON and 3MF, read into the same scene model. **Live demo**: the viewer, in the browser, on your own STEP, glTF, three.js or 3MF files. Nothing is uploaded; it works offline once loaded. Or open one of the demo models there (click a picture):

| | |
|---|---|
|  |  |
| `ohmm-ap242.step`: 71 parts | `rpbp-ap242.step`: 50 parts |




Engine model by Christian Stark.

In more detail:

- **a STEP parser and writer** in pure TypeScript with no runtime dependencies: lossless, schema-aware, tolerant of real-world files, and fast (about 230 MB/s to index, 140 MB/s to decode);
- **a scene model**: the assembly tree with transforms and instancing, names, part numbers, units, colours and layers, as the file states them and as well as they can be recovered from imperfect files;
- **tessellation by WebAssembly backends** behind one per-part interface: our own build of OCCT 8.1 (robust; the default) and truck (Apache-2.0, a seventh of the size and about twice as fast on the parts it meshes, but complete and on OCCT's geometry for only 83.5% of the corpus's parts, and stuck until the timeout on a few), run in a pool of workers with timeouts and an optional cache;
- **exporters** to three.js JSON, glTF/GLB and 3MF, and **importers** of glTF/GLB, three.js JSON and 3MF: our own files with extras come back as the scene they were written from, other files as their own tree of parts and assemblies;
- **two command-line tools**, `step2json` (STEP to JSON, lossless or semantic) and `stepconv` (STEP to three.js, glTF or 3MF).

Version 0.2.0, not yet published to npm: the packages run as TypeScript sources under Node 24 (see Building From Source). A browser-based viewer shows models in three.js, and more input formats are on the roadmap.

```
stepconv bracket.step -o bracket.glb          # STEP to GLB, OCCT, 0.1 mm, all cores
stepconv assembly.stp -o assembly.3mf --3mf-assembly
step2json -m semantic --instances --pretty part.stp -o part.json
```

```
import fs from 'node:fs';
import { exportView, tessellateScene } from 'cad-scene';
import { glb } from 'export-gltf';
import { importStep } from 'step-import';
import { OcctTessellator } from 'tess-occt';

const { scene } = await importStep(fs.readFileSync('bracket.step'));   // the assembly, parts, colours, ...
const occt = new OcctTessellator();
```

```
await occt.init();
const { diagnostics } = await tessellateScene(scene, occt);   // the medium profile: 0.1 mm, 15 degrees
occt.dispose();
const { bytes } = await glb(scene, exportView(scene));     // each part's finest level: here, ours
fs.writeFileSync('bracket.glb', bytes);
```

(Until the packages are on npm, code like this runs inside the repository, where the workspace resolves the package names.)

The decisions behind the design and the lessons learned are in `CLAUDE.md` and its topic notes in `docs/claude/` (the detailed history is in `CLAUDE.md`'s versions at tags `V0.2.0` and `V0.1.0`); `CHANGELOG.md` lists the releases; `step_tutorial.md` is a tutorial on STEP AP203/214/242 in practice.

# 1 Building From Source

On macOS or Linux (the build scripts are bash; Windows is untested).

## 1.1 What You Need First

None of these comes with the repository. `source env.sh` (below) checks every one of them and prints the commands to install or update what's missing, for macOS (Homebrew) or Linux (apt or dnf).

| Tool | Version | Needed for |
|---|---|---|
| Node.js | 24 or later | everything: the sources run as TypeScript through Node's type stripping |
| npm | (with Node) | the workspace and the dev dependencies |
| git | 2.25+ | the fetch scripts |
| curl, unzip | any | the fetch scripts (the NIST corpus packages are zips) |
| CMake | 3.10+ | configuring the OCCT build |
| make | any | building OCCT |
| Emscripten (emsdk) | 6.0.12 | compiling OCCT and our wrapper to WebAssembly (`~/emsdk`, or `$EMSDK`); it brings its own clang, so the OCCT build needs no other C++ compiler |
| a C/C++ toolchain for the host (Xcode command-line tools, or gcc) | any | the truck build: Rust links its build scripts and procedural macros, which run on the host, with the system's linker; also the optional native test of the `VBernstein` patch (`packages/tess-occt/occt_overlay/test/table_test.cpp`, which checks a computed table against the one OCCT stores) |
| Rust, cargo | 1.85+ (edition 2024), with the `wasm32-unknown-unknown` target | the truck backend |
| wasm-pack | 0.15.0 | its `wasm-opt` optimizes the truck module |
| python3 with `venv` | 3.8+, optional | extracting the JT specifications' text; the `VBernstein` patch's tests |
| Typst | 0.15.1, fetched | `scripts/md2pdf.sh` (PDFs of the Markdown documents); not installed on the system: `scripts/fetch-typst.sh` |

| Tool | Version | Needed for |
|---|---|---|
| | | fetches the official binary and the Typst packages it uses, pinned and checksummed, into `third_party/typst/` (`md2pdf.sh` does so itself on first use) |

The project was built and tested with Node 24.5, Emscripten 6.0.12, Rust 1.99.1 and wasm-pack 0.15.0 on an M-series Mac.

## 1.2 Steps

```
git clone https://github.com/martyvona/cad-io-js.git
cd cad-io-js
source env.sh                        # checks the tools; sets up Rust's and Emscripten's environment in this shell,
                                     # and puts node_modules/.bin (stepconv, step2json) on PATH
npm install                          # dev dependencies (TypeScript, three, the validators) and the workspace links
third_party/pull.sh                  # OCCT, stepcode, truck and reference documents, at pinned versions (~400 MB)
packages/tess-occt/build-occt.sh     # OCCT's static libraries for WebAssembly (~6-12 minutes, ~850 MB)
packages/tess-occt/build.sh          # our OCCT wrapper, linked: packages/tess-occt/dist/occt_tess.{mjs,wasm}
packages/tess-truck/build.sh         # the truck backend: packages/tess-truck/dist/tess_truck.wasm
npm test                             # the package tests
npm run typecheck
```

Then the tools run as commands: `stepconv model.step -o model.glb`. `npm install` links them into `node_modules/.bin`, which `env.sh` puts on your `PATH` (at its end, so the dev dependencies' tools there, such as `tsc`, don't shadow ones you have installed). Without `env.sh`, run them as `npx stepconv ...` from inside the repository, or as `node packages/cli/bin/stepconv.ts ...`. The parser, importer, scene and exporters need none of the native builds; only tessellation does (one backend is enough: `stepconv -b truck` without OCCT, the default otherwise).

To run the corpus checks (scripts that run the code over 517 real-world STEP files, against OCCT and the output formats' own validators), also fetch the test corpus (about 1.5 GB):

```
test_corpus/third_party/pull.sh
```

Two of the checks (`cache_check.ts`, `truck_bench.ts`) tessellate per-part subsets of 30 corpus files instead of whole files. Make those once (a few seconds):

```
N=test_corpus/third_party/nist-pmi-models/NIST-PMI-STEP-Files/NIST-PMI-STEP-Files
G=test_corpus/third_party/gkv311-opencascade-dataset/step
for f in $N/*.stp $G/trj2_as1-id-203.stp $G/trj2_as1-tc-214.stp $G/bug21802_as1-oc-214.stp $G/trj7_k1-*-214.stp \
         $G/sp7_04-dx-242.stp $G/bug32049_sp7_10nx_242.stp $G/bug33569.stp; do
  node corpus_checks/make_subsets.ts "$f" test_corpus/subsets/out
done
node corpus_checks/make_subsets.ts $G/Median_cx-fs01_bicycle.stp test_corpus/subsets/out_bike
```

Then run any check, for example `node corpus_checks/writer_corpus.ts`; Corpus Checks below describes each one with its command line.

**The documents** are GitHub-flavoured Markdown, readable as they are on GitHub and in browser Markdown viewers (such as markdown-viewer, with its Mermaid and MathJax options on). `scripts/md2pdf.sh README.md` (or `step_tutorial.md`) makes a PDF with Typst, with no other tools: no pandoc, LaTeX or browser. The PDFs aren't committed: each viewer build makes both, and the deployment publishes them beside the viewer. Diagrams are Mermaid (```` ```mermaid ```` blocks), math is LaTeX (`$...$`, `$$...$$`); how a document is written for the PDF is in `scripts/md2pdf.typ`.

Details: `third_party/README.md` (the pinned dependencies), `test_corpus/README.md`, `corpus_checks/README.md`.

**OCCT build notes:** `build-occt.sh` builds only the `TKDESTEP` and `TKMesh` toolkits and their dependencies, with `-Os` (`-O2`, `-O3` and LTO buy no speed), `-fwasm-exceptions` and `-UOCC_CONVERT_SIGNALS` (without it the link produces

invalid WebAssembly), into `packages/tess-occt/build/occt-Os/`. A build directory can't be moved (CMake keeps absolute paths); delete it and build again. The module is 7.2 MB (2.18 MB brotli). An optional patch, `packages/tess-occt/occt_overlay/`, computes a table OCCT stores and brings it to 1.71 MB brotli; it isn't used (see its README).
**Our patches to OCCT:** `build-occt.sh` builds from `packages/tess-occt/build/occt-src/`, a git worktree of the pinned `third_party/OCCT` checkout that each run resets and patches afresh with `packages/tess-occt/occt_patches/*.patch` (unified diffs, applied in name order; one that no longer applies stops the build), so `third_party/OCCT` stays as fetched and a changed patch rebuilds only the files it touches. Each patch's header says what it fixes, the evidence, and how it's checked. The backend's version names the patches it was built with (`OCCT 8.1 (3d097a0328) + patches 0001-...`), so caches tell patched builds apart. Today: `0001-brepmesh-cone-seam-pcurves` (a cone whose seam pcurves have the slightest slope was left unmeshed). They aren't reported upstream.

# 2 Installing

The packages aren't on npm yet. When they are, this section will say how to install and use them, and the build above will only be needed to work on cad-io-js itself.

# 3 Command-Line Tools

Both are in `packages/cli` (npm name `cad-io-cli`), as `bin/step2json.ts` and `bin/stepconv.ts`. Both read a file or standard input (`-`), plain or gzipped (`.stpZ`, detected by content), write a summary and the diagnostics to standard error (`-q` silences it), and exit with 0 on success, 1 for a usage error, 2 when the input can't be read or the output can't be written. Problems in the file are diagnostics, not failures.

## 3.1 `step2json`: STEP to JSON

```
Usage: step2json [options] <file.stp | file.stpZ | ->

  -o, --output <file>    write here instead of standard output
  -m, --mode <mode>      raw (default): every instance, losslessly
                         semantic: product structure, units, styles, validation properties
      --ndjson           raw mode: one JSON value per line
      --names            raw mode: annotate each record with its attribute names
      --instances        semantic mode: list every product instance with its transform
      --check            semantic mode: check every instance against the schema; adds diagnostics
      --pretty           semantic mode: indent the output
  -q, --quiet            no summary on standard error
  -h, --help             show this help
```

```
step2json part.stp -o part.json                      # raw: every instance, losslessly, streamed
step2json --ndjson --names part.stp > part.ndjson     # one instance per line, attribute names added
gunzip -c part.stpZ | step2json - -o part.json        # standard input (gzipped input is also read directly)
step2json -m semantic --instances --check --pretty assembly.stp -o assembly.json
```

**Raw mode** is the lossless form: the header and every instance, attributes in file order, numbers keeping their integer or real form (`5` against `5.0`), references, enumerations, `$` and `*` explicit. It loads back into exactly the parsed model and writes back to equivalent Part 21; the whole corpus round-trips with no differences. Format:

```
{"format":"cad-io-js raw","version":1,"source":"step",
"header":[{"type":"FILE_DESCRIPTION","attrs":[["a test"],"2;1"]}, ...],
"data":[{"instances":[
{"id":1,"type":"CARTESIAN_POINT","attrs":["origin",[0.0,-1.5,1e-7]]},
{"id":3,"type":"AXIS2_PLACEMENT_3D","attrs":["",{"ref":1},{"ref":2},null]},
{"id":10,"parts":[{"type":"LENGTH_UNIT","attrs":[]},{"type":"NAMED_UNIT","attrs":[{"derived":true}]},
{"type":"SI_UNIT","attrs":[{"enum":"MILLI"},{"enum":"METRE"}]}]},
...]}]}
```

| Part 21 | JSON |
|---|---|
| real `5.` | `5.0` |

| Part 21 | JSON |
|---|---|
| integer `5` | `5` |
| string | `"x"` (decoded Unicode) |
| `$` | `null` |
| list | `[...]` |
| typed `LENGTH_MEASURE(5.)` | `{"type":"LENGTH_MEASURE","value":5.0}` |
| reference `#12` | `{"ref":12}` |
| list of references | `{"refs":[1,2]}` |
| enumeration `.T.` | `{"enum":"T"}` |
| `*` | `{"derived":true}` |
| binary | `{"bin":"0FF"}` |

It is standard JSON (any parser reads it, though `JSON.parse` merges ints and reals); `lexical-json` reads it keeping the distinction. NDJSON: an envelope line with the header, a `{"data":{...}}` line opening each DATA section, then one instance per line. `--names` adds `"names":[...]` before each record's `"attrs"`; loading ignores it.

**Semantic mode** is what the file means, as standard JSON: `{"format":"cad-io-js semantic","version":1, file, roots, products, occurrences, representations, instances?, styles, validation, diagnostics}`. Products carry part number, name, description, version, life-cycle stage, shape representations and documents; occurrences their placement matrices (16 numbers, column-major, units converted); representations their contexts and units; styles the styled items with colours, transparency, overrides and layers; validation the CAx-IF validation properties in SI units. `--check` binds every instance with value checks and adds what it finds to the diagnostics.

On the largest corpus file (59 MB, 537k instances), raw output takes 1.2 s and semantic 0.4 s.

## 3.2 `stepconv`: STEP to three.js, glTF or 3MF

```
Usage: stepconv [options] <file.stp | file.stpZ | -> -o <output>

Formats:
  threejs    .json        three.js JSON Object Scene format 4 (THREE.ObjectLoader)
  glb        .glb         glTF 2.0, binary
  gltf       .gltf        glTF 2.0, JSON with its buffer in a .bin beside it (or embedded: --gltf-embed)
  3mf        .3mf         3MF core, closed meshes (an item per placed part; colours as basematerials)

Options:
  -o, --output <file>      the output file
  -f, --format <format>    the output format, whatever the extension
  -b, --backend <name>     occt (default) or truck
      --coarse, --medium, --fine
                           the meshing profile (default medium), with its deflections:
                             coarse  0.2 mm, 30 deg
                             medium  0.1 mm, 15 deg
                             fine    0.05 mm, 7.5 deg
  -d, --deflection <mm>    maximum distance between mesh and surface, in millimetres (default the profile's)
      --angular-deflection <deg>
                           maximum angle between adjacent mesh normals, in degrees (default the profile's;
                           truck ignores it)
      --tessellated-by-backend
                           AP242 tessellated geometry (triangulated faces, surface sets...) meshed by the backend
                           from its STEP subset, as a B-rep is (default: read as it is; truck reads none)
      --with-edges         write B-rep edges (not 3MF, which has no lines)
      --with-curves        write wireframe curves and points (not 3MF)
      --unlit-lines        edges, curves and points in a flat colour: glTF with KHR_materials_unlit (three.js
                           lines are always unlit; not 3MF)
      --double-sided       render faces from both sides (not 3MF)
      --keep-hidden        glTF: write hidden instances with KHR_node_visibility; 3MF: as ordinary parts
                           (default: left out); three.js always writes them hidden
      --keep-empty         keep assembly nodes with nothing to draw beneath them (default: left out; not 3MF,
```

```
                        which never writes them)
    --extras            our ids for picking and lookup: source ids, properties and face ranges (glTF extras,
                        three.js userData, 3MF metadata); default: left out, as other systems don't read them
    --gltf-compression <draco|meshopt|none>
                        glTF geometry compression (default draco: KHR_draco_mesh_compression, triangles only,
                        lossy; meshopt: EXT_meshopt_compression, every buffer, 16-bit positions)
    --gltf-draco-bits <n>
                        Draco's position quantization, bits over each part mesh's extent (default 14)
    --gltf-instancing   each part placed more than once as one node with EXT_mesh_gpu_instancing (required),
                        out of the assembly tree, losing its occurrence names (edges, curves and points stay
                        a node per placement); mirrored or sheared placements stay in the tree
    --gltf-embed        .gltf: the buffer embedded as base64 instead of a .bin beside it (a third bigger)
    --3mf-colorgroup    3MF: colours as the Materials extension's colorgroup (print colours, opaque) instead
                        of basematerials (display colours)
    --3mf-assembly      3MF: one build item, the assembly as components, instead of an item per placed part
    --3mf-surfaces      3MF: meshes that aren't closed manifolds, and parts the source says are open, as
                        surface objects instead of left out
-j, --workers <n>       parts tessellated at once, each in a worker (default: cores - 1); 0: one at a time
                        on the main thread, without a timeout
    --timeout <s>       per part, in a worker (default 60); 0: none
    --cache <dir>       cache tessellations in this directory
    --cache-size <MB>   the cache's maximum size (default 1024)
    --strict            exit 3 if a part fails to tessellate (the output is written all the same)
-q, --quiet             no summary on standard error
-h, --help              show this help
```

```
stepconv motor.step -o motor.glb                                        # OCCT, medium, faces only, Draco
stepconv --coarse motor.step -o motor.glb                               # fewer triangles (0.2 mm, 30 degrees)
stepconv --fine -d 0.02 part.stp -o part.glb                            # fine's angle, a 0.02 mm distance
stepconv motor.step -o motor.glb --gltf-compression meshopt --extras    # meshopt, with ids for picking
stepconv motor.step -o motor.glb --gltf-instancing                      # repeated parts as GPU instances
stepconv motor.step -o motor.json --with-edges --with-curves            # three.js with B-rep edges and wireframes
stepconv motor.step -o motor.gltf --with-edges --unlit-lines            # motor.gltf + motor.bin
stepconv motor.step -o motor.3mf --3mf-assembly --3mf-colorgroup        # one item, components following the tree
stepconv -b truck -d 0.05 --angular-deflection 15 part.stp -o part.glb  # truck, finer
stepconv --cache ~/.cache/cad-io-js big.stp -o big.glb                  # a second run reads the meshes from the cache
cat part.stp | stepconv - -f glb -o - > part.glb                        # standard input to standard output
stepconv --strict part.stp -o part.glb || echo "some part failed"       # exit 3 if a part has no mesh
```

The summary, for example:

```
1 parts (1 of 1 brep parts tessellated), 1 placed; 5628 triangles; 0.1 MB written
import 0.0 s, tessellation 0.5 s (medium: 0.1 mm, 15 deg), export 0.0 s
0 errors, 0 warnings, 1 notes
```

What each format holds:

| | three.js JSON | glTF / GLB | 3MF |
|---|---|---|---|
| Tree | an `Object3D` tree following the assembly; a `Mesh` per placed part | a node tree following the assembly; a node per placed part (with `--gltf-instancing`, a node per repeated part, under the root) | by default a build item per placed part at its world transform (slicers may rearrange them), the instance path as its part number; `--3mf-assembly`: one item, nested component objects |
| Sharing | geometries shared by the instances of a part (and look) | a mesh per look, shared across nodes | a mesh object per look |
| Units, axes | metres and Y up, in each top object's matrix (a `Scene` holds them; no root group of ours) | metres and Y up, in each top node's matrix (no root node of ours) | the source's unit where 3MF has a name for it (`unit="inch"`), else millimetres |

| | three.js JSON | glTF / GLB | 3MF |
|---|---|---|---|
| Colours | `MeshStandardMaterial`, sRGB | `pbrMetallicRoughness`, linear base colour, alpha blending | `basematerials` with alpha; `--3mf-colorgroup` the Materials extension (opaque) |
| Metadata | names; with `--extras`, STEP ids, properties (the file's too), per-face ranges in `userData` | names; the source's copyright in `asset.copyright`; with `--extras`, the same as three.js in `extras` (with Draco, faces as a `_FACE` vertex attribute) | names, part numbers, the instance path as the item's part number; the core spec's metadata from the source (`Designer`, `CreationDate`, and a 3MF source's `Title`, `Description`, `Copyright`, `LicenseTerms`, `Rating`); with `--extras`, its names in `cadio:path` and the file's properties |
| Compression | none | Draco (`KHR_draco_mesh_compression`, the triangles), or meshopt (`EXT_meshopt_compression` with `KHR_mesh_quantization`, every buffer), or none | deflate (zip) |
| Hidden instances | `visible: false` | left out (or `KHR_node_visibility` with `--keep-hidden`) | left out (or kept) |
| Edges, curves, points | `LineSegments`, `Points` children, on request (always unlit) | `LINES`, `POINTS` child nodes, on request (lit, or unlit with `--unlit-lines`) | none (3MF has no lines) |
| Open shells (meshes that aren't closed manifolds, and parts the STEP file says are open shells) | written | written | left out with a warning (or `surface` objects with `--3mf-surfaces`) |

**Empty assembly nodes**, those with nothing to draw beneath them (only parts that failed to tessellate, external references, hidden parts left out), are left out of three.js and glTF output unless `--keep-empty`; 3MF never writes them.

**glTF instancing** (`--gltf-instancing`) draws each part placed more than once (with the same colours, and visible or hidden alike) as one node under the root with `EXT_mesh_gpu_instancing`, its placements as per-instance transforms; viewers draw them with one call per mesh. Those placements leave the assembly tree, so the tree loses their occurrence names (and the assembly nodes left empty, unless `--keep-empty`); with `--extras`, the instanced node's `extras.instances` lists each instance's tree path, names and occurrence ids, in instance order. The extension is required, since a viewer without it would draw each such part once, misplaced. Edges, curves and points of instanced parts stay ordinary nodes, one per placement under the root: three.js's `GLTFLoader` (0.186) instances only triangles. A mirrored or sheared placement, which the extension's translation, rotation and scale can't express, stays in the tree. Meshes are shared between instances either way, so files barely shrink; the gain is in nodes and draw calls (over the corpus, 2,157 of 5,027 placed parts go into 425 instanced nodes, leaving 3,295 part nodes).

**glTF compression** is lossy either way, and a viewer needs the codec's decoder (three.js: `DRACOLoader`, `MeshoptDecoder`). Draco, the default, quantizes positions to 14 bits over each part mesh's extent (a step of 0.05 mm on a part 785 mm across; `--gltf-draco-bits` for more), normals to 10 bits, and reorders triangles; edges, curves and points aren't compressed. meshopt quantizes positions to 16 bits over each part mesh's extent (its edges, curves and points included; the scale and offset are in the part's node), normals to 8 bits, and compresses every buffer; it reorders triangles only within each face. With `--extras`, faces stay findable for picking: with meshopt as index ranges, as uncompressed;

with Draco as a `_FACE` vertex attribute, the index of the vertex's face in the primitive's `extras.faces` (16 bits, so at most 65,535 faces in one primitive, else the run fails; the corpus's largest has 4,660).

Parts that fail to tessellate don't fail the run; they are in the summary (with `--strict`, exit 3). OCCT's own messages are kept as warnings on their parts, not printed. Over the 517 corpus files, a run with the defaults takes about 200 s in all with 9 workers, nearly all of it tessellation; exporting is 6 s for glTF (with Draco), 15 s for three.js and 33 s for 3MF. The corpus's GLBs come to 34.5 MB with Draco, against 411 MB uncompressed; with edges and curves (which Draco leaves as they are), 90 MB with Draco and 109 MB with meshopt, against 461 MB.

# 4 Viewer

`cad-viewer` (`packages/viewer`, private) is an interactive three.js viewer for debugging, prototyping and inspecting models, not for end users: a single-page web app built with Vite, served by any static server, and usable offline once loaded (a progressive web app); a build is live at https://martyvona.github.io/cad-io-js/. It opens every format through our own packages, off the page's thread: STEP files parsed and meshed (by OCCT or truck in Web Workers, cached in IndexedDB, appearing part by part), and glTF, GLB, three.js JSON and 3MF read by `gltf-import`, `threejs-import` and `3mf-import` into the same scene model, so our own exports' extras (names, STEP ids, faces, hidden and instanced placements, closed parts) come back as they were written. It has an orthographic camera with trackball controls (mouse and touch: the left button turns about the orbit centre, the right pans with the pointer one to one, leaving the orbit centre where it is on the model, the wheel, touchpad, pinch or middle button zooms at the cursor: the point under it stays put, and the orbit centre moves to the nearest thing under the cursor, and stays where it is when there's nothing there (a Mac touchpad's pinch in Chrome comes as wheel events, in Safari as its own gesture events, zoomed one to one with the fingers); a click or tap that picks something makes the point picked the orbit centre, one that misses leaves it), an assembly tree with hiding, picking and selection with details, a CAD rendering style, section planes, and works on phones and tablets. The plan it was built to, in nine phases, and the decisions behind it are in `CLAUDE.md` ("The viewer").

## 4.1 Running It

```
npm run viewer -- start              # the dev server in the background (http://localhost:5180/); or: npx cad-viewer
start
npm run viewer -- open part.glb      # starts it if needed, then opens the file in the default browser
npm run viewer -- shot part.stp -o part.png --view iso   # starts it if needed; renders headlessly to a PNG
npm run viewer -- status
npm run viewer -- stop
```

The dev server serves a **content root**, by default the repository's `test_corpus/`, at `/content/`: `open` takes a file below it (`--content-root <dir>` to serve another directory; stop the server first if it is running with another). `start` and `open` also take `--port <n>`, `--lan` and `--https self|mkcert` (see Phones, Tablets and https). The server's process and URL are in `packages/viewer/.run/server.json`, its output in `.run/server.log`.

## 4.2 Using It

The controls (lil-gui, top right) are in folders, all closed at first:

- **File** opens local files (STEP as `.stp`, `.step` or `.stpZ`, glTF, GLB, three.js JSON, 3MF) with the browser's file picker (pick a `.gltf` together with its `.bin`), or **Load URL…** asks for a URL (the same server, or another that allows cross-origin requests), a path on the page's server (`/content/as1.stp`) or a relative path, which is a path within the content root (`demo/rpbp-ap242.step`; `..` and `.` refused). When a file or URL can't be opened (not found, a server that can't be reached or won't let the page read it, a file that isn't a model, a typed address that's neither a URL nor a content path), an error dialog says why, as does the status line. **Save original…** saves the open model's file as it was read (not an export) wherever you choose, its name suggested (the file's, or the URL's last segment), through the browser's save dialog where it has one (desktop Chrome, over https or localhost), else as a download by that name; a `.gltf` is saved without its `.bin` and images. **Export…** writes the model as it was imported and meshed in another format: a dialog asks for the format (GLB, `.gltf`, three.js JSON, 3MF) and its options, `stepconv`'s with the same defaults (B-rep edges, curves and points, double-sided faces, unlit lines, GPU instancing, glTF's compression and Draco's position bits, the buffer embedded in a `.gltf` or saved as a `.bin` beside it, 3MF's colours, build and open meshes, hidden instances

kept, empty nodes kept, our extras), and whether to apply the viewer's hiding and opacity (each placement shown or hidden as the viewer draws it, its opacity and its faces' as materials; a face hidden in a placement still shown, and triangle edges, can't be written, and the status line says so); then the save dialog (or a download), its name the model's with the format's extension; a `.gltf` with its buffer beside it asks for the `.bin` in a second step. The file is made in the import worker; the view is paused meanwhile (its meshes are lent to the worker). When the model has transforms the chosen format can't hold (glTF's nodes can't hold a shear), the dialog says so, and the export's warnings in the status line say what was written without them. If the worker stops answering (it sends a heartbeat each second while it exports; a minute without one), the export fails, said in a dialog, and the model is loaded and meshed again as it was, the view kept. The choices are remembered in the browser. **Screenshot…** saves an image (PNG or JPEG) of the model alone, or of the whole window with the tree and the controls. The model alone is the view cropped to the on-screen box of the model's bounding sphere, cut to what's visible between the tree panel and the controls, with a transparent background as an option for PNG. The whole window is redrawn from the page (any browser), or taken by the browser's own tab capture where it has one (desktop browsers; the browser asks first). The dialog's width starts at the size on screen in CSS pixels, and the height follows in proportion; the format and what's shown are remembered. **About…** says the version (also in the controls' title: the root `package.json`'s, and in a build such as the deployment the commit's first 7 hex digits, `0.2.0-fd8a1c1`), who made the viewer (Marsette Vona, then Claude), its license and where its source is (and in a build, **PDF documentation and STEP tutorial**: this README and `step_tutorial.md` as PDFs beside the page, made by each build), and the current model: its file's name with its author where the file names one (`VW.step (Christian Stark)`; not a placeholder such as `Author`), its objects and its triangles. Its **details** link opens **Model Details** over About: what its transforms do beyond turning, moving and scaling evenly (mirrored, scaled unevenly, sheared; and its placed parts', where composition differs), everything the file says of itself (see Scene Model, file metadata; for our files with extras, the source's), read-only, its text selectable for copying, closed by its x or Escape. **Load demo model…** closes About and lists the demo models (each with its size: `8.2k`, `23.4M`), fetching their catalog (`content/demo/catalog.json`) from the server each time it opens; **Load** loads the one chosen, as `demo/<path>` in Load URL… would, and it's chosen again next time. When the server has another version than the one running (a new deployment), a badge (circling arrows) appears after the version in the controls' title, its tooltip saying which version and to reload to update; clicking it opens About, which says the same with a **Reload** button. Nothing is polled: the page fetches the server's `version.json` (beside the page in every build) only when its tab becomes active again (after another tab, another window or the browser minimized), never at load, and About's **Check for updates** fetches it when pressed. Seeing a new version, it has the service worker fetch the new build at once (the badge dim meanwhile), so that one reload shows it. Offline, or with no answer within 10 s, a check from the badge does nothing and About says why. A download that fails (the network lost midway, a file missing) changes nothing, since the browser discards a service worker whose files couldn't all be fetched: the badge goes (a later check tries again), and if About started or joined it, About says so. Files can also be dropped on the page, and `?model=` in the address loads one at startup, read as Load URL… reads what's typed (`?model=demo/rpbp-ap242.step`, `?model=/content/as1.stp`, `?model=https://...`; `?url=` is the same, for older links, and `model` wins if both are given). The content root is `content/` beside the page: the dev server's `/content/`, and in the GitHub Pages deployment the demo models (`content/demo/`).

- **View** (each new model starts looking along the iso direction, fitted, however the last was turned; **Mesh again** keeps the view) has zoom all (fits the model's bounding sphere, so the framing is the same from any direction; while a model loads, the view zooms to it as soon as the model is read, before any mesh, and then out as meshes arrive, at most once a second, until you move the view), **Zoom selected** (fits the selection's bounding sphere; disabled with nothing selected), **Copy link to this view** (see Driving It), **Axes** (a gnomon in the 3D view's lower left, just above the status line (on phones, beside the tree while it's open over the view), turning with the model: the file's own axes, X red, Y green, Z blue, each labelled with its letter beyond its tip on a translucent, round-cornered backing (labels that would overlap each other or the point where the axes meet are nudged sideways, gliding there rather than jumping, and when the view flies to another the labels move smoothly to where they'll end up), an axis along the line of sight shows where the axes meet as a ringed dot when it points at you (an arrow's head) or a ringed cross when it points away (its tail), the ring filled with the view's background so nothing behind shows through, fading in as the view comes within 10 degrees of it; so a glTF or three.js file shows its Y up where the viewer's Z is; it turns about the

centre of the smallest sphere holding its origin and three tips; clicking a letter looks along that axis with it pointing at you (Right, Back or Top for STEP's x, y and z), or away from you if it already does, and clicking where the axes meet looks iso, unless the view is iso already (a finger's tap reaches further than the drawn boxes: anywhere within 20 pixels of a target's centre, the nearest one); shown by default, remembered in the browser, left out of pictures of the model alone, and off in `cad-viewer shot` and browser-check pages unless `?gnomon=1`), the views along the axes (front looks along +Y with Z up; top has +Y up, bottom -Y; each turns the camera about the point it orbits, keeping the zoom and pan); zoom all, zoom selected and the views move the camera there over a second, easing in and out (dragging or zooming stops it where it is); zoom all and zoom selected fit what they show in the width between the tree and the controls, centred there, however folded the controls are (the point the camera turns and zooms about is there too), except where the controls would leave less than half the view (phones), when they use the whole view; and full screen (disabled where the browser can't, as on iPhone), and **Theme**: **Auto** (the default: the system's light or dark setting, followed as it changes), **Light** or **Dark** (remembered in the browser); **Custom background** with **Background**, a colour picker, for a 3D view background of your own instead of the theme's (the picker is enabled only while it's on; both remembered in the browser); then **Details** (see **Selection**), and hiding and transparency (see **Tree**).

- **Tree** (**Show the tree**, **Faces, edges, curves**, **Expand all**, **Collapse all**; hiding and transparency are at the end of **View**): the tree (on the left, from when a model loads, unless you've closed it; over the view on phones, closed at first) shows the assembly from the file's own roots (no node for the file itself; a file with several roots shows them all at the top), each placement of a part separately, as placed, down to parts by default. "Faces, edges, curves" shows below each part its faces, one node for its B-rep edges, its curves and its points (for glTF and three.js files without our extras, its faces are the mesh's primitives or groups, numbered from 0). Click, ctrl/cmd-click and shift-click select rows (the arrow keys move). Hiding and transparency are overrides on the tree's nodes, each holding for everything below the node that doesn't override it in turn: **Hide selected** and **Show selected** set it on the selection (on its assembly instead where all of an assembly's children are selected), leaving what was set below as it is, so a part can be shown inside a hidden assembly and stays shown when the assembly is hidden again; **Show all** clears every hiding (also what the file hid); **Hide others** hides everything but the selection and what holds it, and **Show others** shows those again (the selection keeps its own hiding). **Make transparent** gives the selection the **Opacity** set beside it (the slider only sets what the next press writes), and **Restore opacity** puts back the file's, also inside an assembly made transparent; edges, curves and points keep theirs. A node with an override of its own has an eye after its name in the tree (crossed out when it hides the node; its tooltip says what's overridden). Right-click a row (a long press on a touch screen) for **Hide** and **Show**, which set that node's own visibility at once (as Override Appearance… would, the rest of its override kept), **Remove Appearance Override** (for a node with one), which clears it at once, and **Override Appearance…**; when the row right-clicked is selected along with others, Hide, Show and Remove act on the whole selection and say how many (`Hide 5`, `Hide 8.5k` past 999; Remove counting those with an override, and offered if any has one), and Override Appearance… still on the row right-clicked alone. Right-clicking a row moves the tree's focus (the outlined row the arrow keys move from) to it, leaving the selection as it is. a panel that floats over the page without blocking it (the view and the tree keep working; drag it by its title, close it with its x or Escape; right-clicking another row, or selecting one node alone, shows that node in the same panel: it shows whichever came last, and selecting several nodes or none leaves it as it is): **Visibility** (Inherit, Hide, Show), **Opacity** (Inherit, Transparent, Default: the file's own) and **Triangle edges** (Inherit, On, Off: where nothing sets them, Rendering's Triangle edges decides), each Inherit when the node has no override of its own, with a slider and a box for the alpha, which take input only while Opacity is Transparent and otherwise show the opacity the node draws with (inherited, or the file's); every change takes effect at once, on exactly that node's override (there's no OK or Cancel). Dialogs can be dragged by their title. Hidden rows stay in the tree, greyed, as does everything below them. Hiding a face, or making it transparent, does so in that placement of its part only. The handle on the tree's right edge drags its width (remembered in the browser; on touch screens a wider grip, always shown); a double click on it goes back to the default. Names are never cut short: when they don't fit, the tree scrolls sideways. Faces that failed to mesh, and parts that did (an error or a timeout), are shown in red with a broken icon, and in the 3D view their faces' outlines, taken from the STEP file, are drawn in red where the faces should be; clicking an outline picks its face. Each face's mesh is judged against the face's area on the exact geometry, which OCCT measures as it meshes: red for a face with no

triangles, one whose mesh is much larger than the face, or one OCCT can't measure soundly; the same in the tree, but not outlined in red, for a face whose mesh falls short of its area (part of it missing), and for a part only some of whose faces failed; the red outlines are drawn in front of the black edges along them. Details says which, with the face's area and its mesh's.

- **Selection**: a click or tap on the model (one that doesn't move the view) picks what's under it at the **Pick** level: an assembly, a part (the default), or a face, B-rep edges, curves or points (which shows them in the tree). Ctrl/cmd/shift-click adds or removes, and the **Multi-select** switch does the same for touch; a click on nothing clears the selection. What's hidden, or cut away, isn't picked. The face under the pointer decides, except that its own part's edges, curves and points within 4 pixels win over it (so they can be picked), as do wireframes in front of it; off the model, an edge or point within 4 pixels is picked. The selection is the tree's too, and is tinted orange (a selected face by an overlay); **All instances** also tints, in blue, every other placement of the selected parts. **Details** (in **View**, on by default) shows what's selected: for one node its name, its place in the tree and, for a part, which of its part's placements it is ("Placement 3 of 8"); then the triangles and their area, the line segments (edges, curves) and their length, and points, in millimetres (in the view's top left; it fades 5 s after it last changed while it says "Nothing selected."). **Select parent** selects the selection's parents, **Select none** nothing, and **Select all** everything a click could select at the **Pick** level and isn't hidden: every part, every part's assembly, or every face, edges, curves and points; **Invert selection** selects what Select all would, less what's selected now. Every selection made outside the tree (picks, these buttons, Select by ID) shows in the tree, its branches unfolded. **Select by ID…** asks for STEP instance ids, typed or pasted (separated by spaces, commas or lines, each `#123` or `123`), and selects what has them: an assembly by its occurrence or product, a placed part by its occurrence (NAUO), every placement of a part by its product or its shape representation, a face by its own (turning on the tree's faces), each shown selected in the tree, its branches unfolded; the status line says which weren't found. Details lists the selection's STEP ids too (cut short past about 50 characters). The status line (bottom left) says what a load is doing (importing, with a STEP file's phases: reading, reading its structure, making parts with a percentage, assembling; then the parts shown, edges found and parts meshed, counted) and fades 5 s after it last changed.
- **Section**: up to three planes, one across each world axis, each at a position along it (the sliders span the model) with the side above it cut away (**Flip** for the side below). Where they cut a closed part, a cap shows the cut, hatched with lines at an angle and spacing of the part's own (one of 6 angles and 3 spacings, the widest 1/60 of the model's size), fixed on the model as if painted on it, in a **Cap style**: **Part colours** (each part's colour, lines a darker shade of it, as Onshape draws them; the default) or **Drawing** (beige, two beiges alternating between part colours, black lines, as in a drawing). **Hatch width** sets the lines' width in pixels, the same at any zoom (1.33 by default, as the cadrender prototype draws them). A part is closed as the file says (STEP: every face of the part in a closed shell, a solid's; not if one is in an open shell), or where it doesn't say (glTF and three.js files, a STEP geometric set's surfaces), as its triangles close up (a glTF mesh's primitives are one part, as Onshape writes a primitive per face), up to gaps narrower than 0.1 % of the part's size. Picks skip what's cut away, and a pick on a cap picks the part it cuts (at the point on the plane; `cadViewer.pickAt` says `cap`), as does the zoom at the cursor. Each model loaded starts with every plane off.
- **Rendering**: Phong faces with their edges, the edges as **Edge lines Fat** (three.js's fat lines, **Edge width** CSS pixels wide, 1.5 by default) or **Thin** (WebGL's own, one device pixel; the default on phones and tablets, whose GPUs fat ones would cost more), **Shininess** and **Specular** (the faces' highlights), the **Ambient light** and the **Headlight** (a light from over the viewer's shoulder, turning with the view), **Reset lighting to defaults** (those four only), **Triangle edges** for inspecting the tessellation (every triangle's edges over the faces: off, the selection's only, following it, or all; hidden parts and faces and section cuts apply to them too; `&triangleEdges=selected` or `all` in the address), ambient occlusion (N8AO, its radius in screen pixels as models are viewed at every scale; intensity, radius and quality adjustable) and SMAA antialiasing, each of which can be turned off. Every format gets the same Phong materials (glTF's and three.js's own materials give only their colour and opacity; textures aren't read, and a warning says so). Colours: STEP's are sRGB (the convention every writer follows), glTF's linear as its specification says, except Onshape's glTF files (their `asset.generator`), whose colours are the same sRGB numbers as its STEP files' and are read so, so a model looks the same either way, and as in Onshape. Transparent materials are drawn as three.js does, sorted whole objects at a time.

- **B-Rep Meshing** starts with **Device**: the kind of device the viewer thinks it's on, **Auto (phone)**, **Auto (tablet)** or **Auto (desktop)**, from the screen's size (not the window's), the primary pointer and the touch points, which you can override (remembered in this browser) with **Phone**, **Tablet** or **Desktop**. Phones and tablets mesh **coarse** by default, with at most 3 workers; desktops **medium**, the library's default. Then the backend (OCCT, the default, or truck), **Tessellated geometry** (AP242's triangulated faces and surface sets: **Read as it is**, the default, or **By the backend**, meshed from the STEP file as a B-rep is), the **Profile** (coarse, medium, fine, as `stepconv`'s, or custom: moving the deflection or angle makes it custom; **Defaults** goes back to the device's profile), the deflections, **Timeout** (the seconds a part may take before its worker is stopped and the part counted failed, 60 by default, 0 for no limit; a part that timed out is tried again with a longer limit, even from the cache), and the IndexedDB cache (on by default; it can be cleared), and **Mesh again** meshes the file again with them, keeping the view: the camera, section planes, what's hidden and selected, and the tree's expanded rows and scroll position (the status line says how many hidden or selected nodes weren't found again, such as the faces of a part that failed this time; their paths are in the console). Files are always read (and STEP files meshed) in a worker, off the page's thread. Once a file is meshed, its warnings and errors (from reading it and from meshing: OCCT's messages, faces that failed to mesh, shells turned over, parts that failed), if any, are listed in a dialog, errors first, each with the STEP instance it came from (at most 500; all of them in the console, and their count in the status line); **Copy IDs** copies those instances' ids (`#12 #345`), for Select by ID.

## 4.3 Driving It

**Links**: a page's address carries the view, so **Copy link to this view** (View) gives a link that shows someone else exactly what you see, and a link can be written by hand; the model is linked as `model=`, by its path within the content root where it's below it, so the link works wherever the viewer is served with that content (the dev server, the deployment's demo models), else by its path on the server or its URL. The view follows after `#`, which browsers never send to the server, so a link can be as long as a view needs (servers refuse long queries: GitHub Pages one over about 8 KB, the dev server about 16 KB); the same parameters in the query, as older links have them, are read too (the fragment's first where both give one); `deterministic` and `controls=0` are the page's own options and go in the query. All optional, applied once the model has loaded:

| Parameter | Meaning |
|---|---|
| `view=iso` | an axis view, turned about the target (at once): front, back, left, right, top, bottom, iso |
| `camera=px,py,pz,tx,ty,tz,ux,uy,uz,h[,x,y]` | the camera exactly: position, target, up, the view's height in world millimetres, and the pan (millimetres along the view's right and up) |
| `cut=x:12.5`, `cut=y:-3:flip`, `cut=none`; `caps=0` | section planes (repeated; `flip` cuts away the side below); caps off |
| `ao=0`, `smaa=0`, `edges=0`, `shininess=50`, `aoIntensity=3`, `aoRadius=16`, `aoQuality=Low` | rendering |
| `tree=0`, `detail=1`, `pick=assembly\|part\|detail`, `instances=1` | the tree panel, its faces/edges/curves, the pick level, all instances |
| `hide=<path>`, `show=<path>`, `alpha=0.3:<path>`, `alpha=default:<path>`, `tri=on:<path>`, `tri=off:<path>` | (repeated) the appearance overrides, exactly these: nodes hidden, nodes shown inside something hidden, opacities (`default` the file's own) and triangle edges, each holding for what's below the node unless overridden there |
| `select=<path>` | (repeated) these selected |
| `deterministic` | one pixel per CSS pixel whatever the screen, so screenshots compare across machines |
| `controls=0` | the controls hidden (`cad-viewer shot` asks for it: its picture is of the view alone, so fits centre in the whole view) |

What can't be applied (an unknown path, a malformed value) is said in the status line. Nodes are named by **paths** that stay the same from one load to the next: occurrence names from one of the file's roots down (a placed part's its placement's, else its product's, else its part's: one node, so `as1/l-bracket-assembly:2/nut_3`, not `.../nut_3/nut` as before 2026-11-01), joined by `/`, each followed by `:k` for the k-th sibling of that name (k > 1), with `%`, `/` and `:` in names escaped as in URLs; below a part, `face#<STEP id>` (in glTF and three.js files without our extras, `face#<k>`, the primitive's or group's number), `edges`, `curves`, `points`, as in `as1/plate_1/face#123`.

**`window.cadViewer`**, for scripts and the browser's console (`src/automation.ts`; version 5, which changes only by additions: version 1 had centring, removed 2026-10-30; in version 2 a state's `hidden` was every hidden node, and a face's hiding its part's in every placement, until hiding became the overrides below, 2026-10-31; in version 3 a leaf product's placement was two nodes, its occurrence and its part, until 2026-11-01; in version 4 the tree's top node was the file, so every path began with its name, until 2026-11-06): `ready` (a promise: the start, `model=` and the view in the address, done), `load(url)` (resolves with the load's report once every part is meshed), `idle()` (nothing loading and the last frame drawn), `state()` (the view as the link has it, plus the model's name, URL, counts, bounding sphere and parts), `apply(state)` (returns what couldn't be applied), `link()`, `nodes()` (paths, kinds, names, hidden), `select(paths, {add})`, `hide(paths)`, `show(paths)` (each returns the paths that name nothing), `showAll()`, `view(name)`, `zoomAll()`, `zoomSelected()`, `pickAt(x, y)` (fractions of the view from its top left: the path and the point), `screenshot()` (a PNG data URL, drawn then). The controls follow whatever it does.

**`cad-viewer shot <file>`** renders a file below the content root headlessly (Chrome, `$CHROME` or `--chrome`; through the page's `cadViewer`) to a PNG of the view only (the tree closed, unless `--state` opens it; `?deterministic`): `-o <png>`, `--size <w>x<h>` (default 1200x800), `--view <name>`, `--state "<query>"` (more of the view, as in a link: `"cut=x:10&hide=as1/plate_1&ao=0"`), `--json <file|->` (the page's state, the view's link and its status line), `--theme dark|light|auto` (dark by default; auto is headless Chrome's own preference, light), `--background <#rrggbb>` (the view's background instead of the theme's: the pictures above are on `#ffffff`, trimmed). It starts the dev server if needed and leaves it running; it prints the PNG's path, size, the model and its triangles, and the link.

## 4.4 Recording and Replaying

To reproduce a bug, record what you do: **Recording**, **Record** (or `?record` in the address, from the page's start), then **Download recording**: a JSON file of timed events (`t`, milliseconds from the start): pointer presses, drags and releases on the view and wheel turns, with their positions in CSS pixels from the view's top left; each pick (its pixel, the node picked and the point); selections (from the tree, a pick or the API); controls changed and buttons pressed; the camera after each drag or zoom; files opened (a URL with the size and SHA-256 of what it gave, or a local file's name, size and SHA-256: never its contents); calls to `cadViewer`; the status line, errors, the view's size; and the whole view at the start and at the end (camera, the planes cutting, caps, every rendering setting, the tree, the pick level, the appearance overrides (hidden, shown, opacities) and the selected nodes, faces' included whether or not the tree shows them, the meshing settings, and the model with the settings it was meshed with; the address's view too, for a recording begun with the page), with the browser, the window and its pixel ratio, and the tree panel's width (replay sets it, so the view is as wide as it was). Replay meshes the model as it was meshed then (its settings set before it loads, whatever this browser's device would choose; the device remembered here is left alone), then sets the view as it was; not recorded: planes that are off, the theme, the Details and Multi-select switches, which folders are open, and the tree's expansion and scroll.

**Replay recording…** asks for the recording alone. If the model open is the one the recording was made with (the same SHA-256), the replay starts; else a dialog names the recorded model (name, size, SHA-256, and its URL if it came from one) and offers **Open file…** and **Open URL…** (the recorded URL filled in). A file whose hash differs isn't loaded: an error dialog shows both files' names, sizes and hashes, and the replay stops. Then it replays in the page at the recording's pace, so you can watch it: each step waits for its recorded gap after the last (pauses longer than 3 seconds are cut to 3), drags turn the camera over the drag's time with a marker where the pointer was pressed, each control is shown as it changes (its folder opened, the control flashed; the folders close again at the end), and **Stop replay** ends it. Order comes first: a step starts only once the last is done (a load meshed, Mesh again finished), so a step slower than it was makes the replay slower, never out of order. Replayed are controls, selections, loads, the camera after each drag or zoom (pointer

moves themselves aren't replayed: the controls capture the pointer), and each pick at its recorded pixel through the view's own picking, compared with what was picked then. It reports divergences: a pick landing elsewhere, a file not given, a control gone, a view of another size at a pick, an end state that differs. `cadViewer.record(on)`, `recording()` and `replay(recording, { files, urls, openModel, realtime, maxPause, signal })` do the same from scripts (`openModel(model)` opening each recorded model, as the dialogs do; as fast as the viewer allows unless `realtime`, as are `cad-viewer replay`'s).

**`cad-viewer replay <recording.json>`** replays headlessly at the recorded window's size and pixel ratio, finds local files below the content root by name and size, writes the view at the end (`-o`, `--json` for the report and state, `--theme` as for `shot`), prints the divergences, and exits 3 if there were any. Timing-dependent behaviour (a click while a model is still meshing) replays only approximately, as each step waits for the viewer; the timestamps say when things happened.

## 4.5 What the Viewer Does with Files

**STEP files**: a STEP file is parsed in a worker, so the page stays responsive (the bicycle's longest task on the page: 64-116 ms, against 410 ms parsing on the page). Its assembly appears at once and its parts as they are meshed, by a pool of workers (one less than the cores, at most 8; at most 3 on phones and tablets, as **Device** says); the view zooms to the first part, and to the whole when the last is done, unless it has been moved meanwhile. Opening another file while one is meshing stops the meshing. Parts are drawn with their B-rep edges (in glTF and three.js files, ours written with or without extras, their `edges`); parts without any get edges found from their triangles, where faces meet at more than 10 degrees.

**Units and axes**: the world is millimetres with Z up. STEP files load as they are (most CAD systems model with Z up, some with Y); glTF and three.js JSON files (metres and Y up at their root, ours included) are turned and scaled to it by their importers; 3MF files are Z up already (the build platform's) and scaled from their unit. The page draws only when something changes, at most 30 frames a second.

**Phones and tablets**: on a narrow screen the tree and the controls start closed (each opens over the view; the tree at its narrowest, 140 pixels, by default, dragged by its handle to as little as 40 pixels from the screen's edge); one finger rotates, two zoom and pan, a tap picks; the page doesn't scroll, bounce or offer its long-press menu over the model; meshing uses at most three workers. iOS (Safari's engine, whichever browser) is supported as far as it goes, and has no full screen on iPhone.

## 4.6 Building and Deploying

```
npm run build -w cad-viewer          # writes packages/viewer/dist/ (about 12 MB, most of it OCCT's module)
```

`packages/viewer/dist/` is the whole app, to copy to any web server, at any path (its URLs are relative: the browser check serves it from `/viewer/` by a plain static server). The server needs nothing special beyond content types: JavaScript for `.js` and `.mjs` (module scripts refuse anything else) and `application/wasm` for `.wasm`. A build has no `/content/`: it loads files by URL (its own server's, or another's that allows cross-origin requests) or from the device.

**GitHub Pages**: `npm run deploy-viewer` (`scripts/deploy-viewer.sh`) builds the viewer and force-pushes the build as the only commit of the `gh-pages` branch, so it is served at `https://<owner>.github.io/<repository>/`. The first deploy creates the branch; then, once, in the repository's Settings, Pages, choose to deploy from the `gh-pages` branch, its root (it can't be chosen before the branch exists). It deploys only a clean commit that is already on the remote (so the source `licenses.md` points to is public), with the backends' modules as built on this machine (git doesn't hold them); `--dry-run` builds and commits in a temporary directory without pushing. Each build also holds this README and the STEP tutorial as PDFs (`README.pdf`, `step_tutorial.pdf`, made with `scripts/md2pdf.sh`, which fetches Typst the first time; linked from About, not precached), so they're published on the `gh-pages` branch, its one commit, rather than committed to `main`. The demo models in `test_corpus/demo/` are copied into the build's `content/demo/` (not precached by the service worker), so `https://martyvona.github.io/cad-io-js/?model=demo/rpbp-ap242.step` loads one, as does `demo/ohmm-ap242.step` in **Load URL…**, or About's **Load demo model…**. That lists the models in `content/demo/catalog.json`: `node scripts/demo-catalog.ts` writes it (every file below `test_corpus/demo/` the viewer reads, by

its extension, with its path and size in bytes; not dot files), and the deploy commits it when it changes (and pushes it). The deploy takes the demo models as committed, and refuses to run while any are added, changed or removed but not committed. **Demo models alone**: `npm run deploy-demo` (`scripts/deploy-demo.sh`) updates the catalog, commits `test_corpus/demo/` (only it) when anything there changed and pushes it, then force-pushes `gh-pages` with its `content/demo/` replaced, the app as deployed (its version unchanged, so viewers see no update badge); it refuses while the branch has other unpushed commits, and `--dry-run` builds the site in a temporary directory from the demo directory as it is, committing and pushing nothing.

**Offline**: a build is a progressive web app. Its service worker (in builds only, and only over https or on localhost) caches everything the app needs on the first visit, the OCCT and truck modules included (about 12 MB), so the device can use it with no network afterwards, opening local files and meshing STEP. A new build is picked up at the next load when online, or, once the update badge has appeared and turned bright, at the first reload. Browsers can also install it as an app.

**Licenses**: a build bundles OCCT (LGPL-2.1 with the Open CASCADE exception, in `tess-occt`'s module), truck, three.js, Draco's and meshoptimizer's decoders, Headless Tree and N8AO; `dist/licenses.md` says what's under which license, with the license texts (the npm packages' as Vite collects them, then ours and OCCT's in full). Serve it with the app.

## 4.7 Phones, Tablets and https

`--lan` makes the server listen on every interface, and `start` prints its LAN addresses: open one on the phone or tablet (same network). Over plain http, which is the default, a LAN address isn't a *secure context*, so the browser withholds what needs one: the service worker (so no offline use), the File System Access picker (the plain file picker still works) and Web Crypto (our own SHA-256 stands in, so tessellation and its cache still work). For those, serve https:

- **`--https self`**: a self-signed certificate made on first use (`@vitejs/plugin-basic-ssl`). Each browser warns once and must be told to proceed (Chrome: "Advanced", then "Proceed"); Chrome still refuses to register a service worker under a certificate error.
- **`--https mkcert`**: a certificate from a local certificate authority that each device trusts; everything works, at the cost of installing the authority on each device. Once, on the computer running the server:

```
brew install mkcert                  # or see https://github.com/FiloSottile/mkcert
mkcert -install                      # trusts the local authority on this computer
mkdir -p packages/viewer/.cert
mkcert -cert-file packages/viewer/.cert/cert.pem -key-file packages/viewer/.cert/key.pem \
  localhost 127.0.0.1 "$(hostname)" "$(ipconfig getifaddr en0)"   # the LAN address (macOS); add any others
```

  Then on each phone or tablet, install the authority's certificate, `rootCA.pem` in the directory `mkcert -CAROOT` prints (send it by mail, AirDrop or a USB cable). **Android**: Settings, Security, Encryption and credentials, Install a certificate, CA certificate (the menu names vary by maker). **iOS and iPadOS**: open the file to download the profile, install it in Settings, General, VPN and Device Management, then turn on full trust for it in Settings, General, About, Certificate Trust Settings. If the computer's LAN address changes, make the certificate again with the new one. The `.cert` directory is ignored by git.

## 4.8 Checks

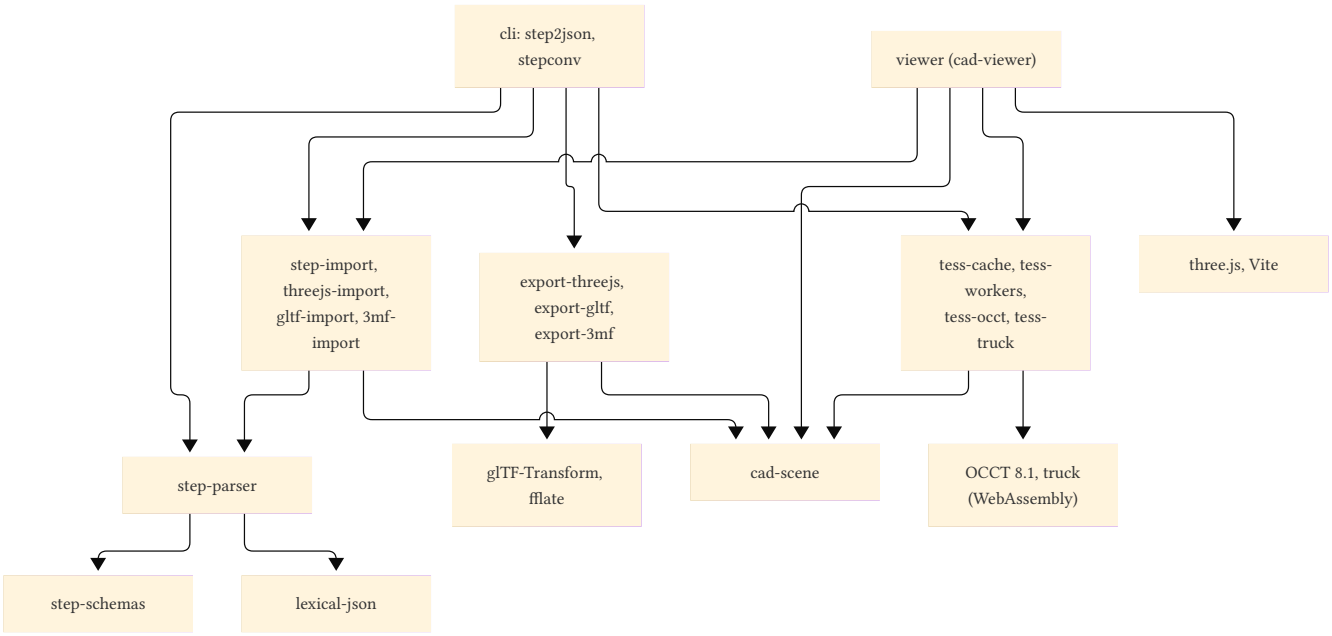
Its unit tests run with `npm test`. `node packages/viewer/test/browser-check.ts` (by hand; needs `tess-occt` built and the corpus) loads a corpus assembly in headless Chrome as STEP (meshed in the page's workers) and converted to GLB with Draco, glTF with meshopt, three.js JSON and 3MF (each with and without extras), served by the dev server, comparing the triangles, the B-rep edges' segments, and the world box in millimetres with what the scene says; it also checks edges found for a GLB without them, a phone (emulated: closed panels, a tap with touch events), and camera flights (zoom all, zoom selected and the views eased over a second, a view turning about the target, input stopping a flight), fitting beside the controls (open and folded, in a wide window and the whole view in a narrow one; picks checked against what's drawn there too), zooming while the bicycle (`Median_cx-fs01_bicycle.stp`) loads (fitted to its estimated sphere before any mesh, then only out, at most once a second, everything inside the sphere at the end; not after the view is moved), replacing the bicycle while it meshes (the meshing stops, the cache answers the next load,

truck meshes when chosen); `SHOTS=<dir>` saves a screenshot of each page, `ONLY=<check>,...` runs only the checks named (each check's name and time go to standard error as it runs), `EXTRA=<file>,...` also loads those files and prints what the page reports (the bicycle: 3.8 s from opening to the last part, 875,317 triangles, in headless Chrome on an Apple M4, cache empty), the automation interface (nodes by path, select, hide, a pick, a screenshot; a view taken as a link and opened in another page gives the same state, and the controls follow), `?deterministic` (one pixel per CSS pixel at a screen scale of 2, screenshots that repeat) and `cad-viewer shot` (a PNG of the size asked for, the scene's triangles in its JSON; dev server only), recording (a session driven by real input and controls, recorded with timed, pixel-placed events, replayed in a fresh page with nothing diverging; a tampered pick, a wrong file said; `cad-viewer replay`), and `BUILD=1` checks a build (`vite build`) instead of the dev server, served at `/viewer/` by a plain static server (the models at `/content/`), what the build holds (one Draco decoder, `licenses.md`, `version.json` and its absence from the precache), a new deployment (the build on the server changed under a controlled page: the update seen, ready once the new service worker has taken over, and shown after one reload), and then offline use: the server stopped, the page reloaded from its service worker's cache and the assembly meshed.

# 5 Architecture

## 5.1 Package Dependencies

An npm workspace of 17 packages. Arrows point from a package to what it uses.

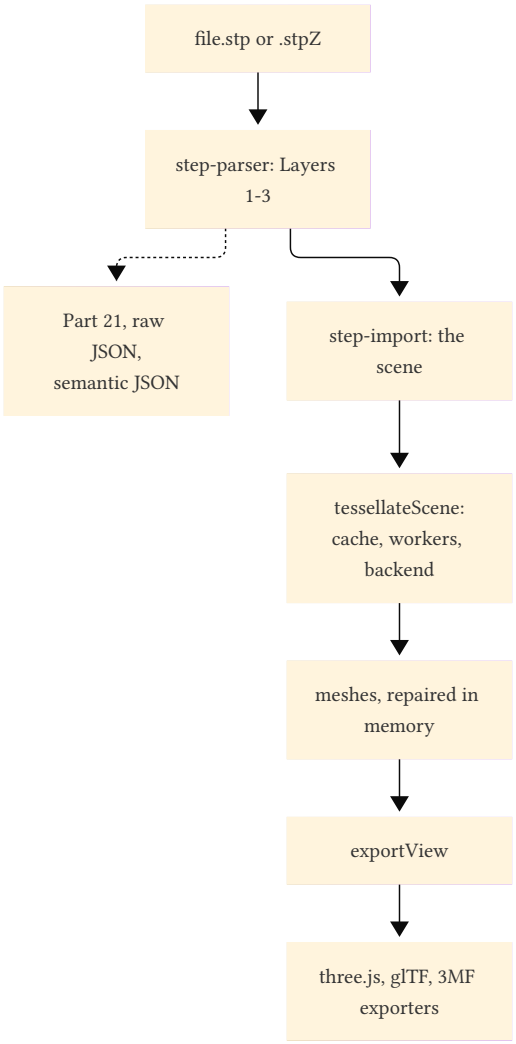



- **`step-parser`:** Layers 1 to 3 of STEP reading, the Part 21 writer and raw JSON. **`step-schemas`:** the EXPRESS dictionaries, generated from stepcode's EXPRESS schemas. **`lexical-json`:** JSON that keeps integers and reals apart.
- **`cad-scene`:** the scene model, the tessellator interface, `tessellateScene`, `exportView`, the mesh helpers, and `sceneFromTree`, the core of the importers of glTF and three.js JSON (and of 3MF files with our extras). **`threejs-import`:** three.js JSON to a scene. **`gltf-import`:** glTF and GLB to a scene. **`3mf-import`:** 3MF to a scene.
- **`viewer`:** the interactive viewer (see Viewer), on the importer, the tessellation packages and three.js.
- **The backends:** `tess-occt` is our build of OCCT 8.1 (`third_party/OCCT`) as WebAssembly; `tess-truck` is our Rust wrapper over truck (`third_party/truck`), likewise.
- **No runtime dependencies** in `step-schemas`, `lexical-json`, `step-parser`, `cad-scene`, `step-import`, `threejs-import`, the tessellation packages and `export-threejs`. `export-gltf` and `gltf-import` use glTF-Transform (MIT) and the Draco (Apache-2.0) and meshoptimizer (MIT) codecs, and `export-3mf` and `3mf-import` fflate (MIT).
- **The EXPRESS dictionaries** are generated by `step-schemas`' own EXPRESS parser from the AP203, AP214 and AP242 schemas in stepcode (`third_party/stepcode`, fetched by `third_party/pull.sh`), and committed as JSON

(`packages/step-schemas/dict/`). Generating them is a development step (`npm run gen -w step-schemas`, needed only to change them); stepcode is needed neither to build nor to run. The dictionaries themselves are needed at run time, and ship in the `step-schemas` package: Layer 2 (`openModel`) loads the one a file's `FILE_SCHEMA` names, plus AP242 as the fallback for unknown entities, by dynamic JSON imports, so only those are fetched (in a browser, a bundler makes them separate chunks). Everything that binds names needs them: the semantic layer, the importer, `step2json -m semantic` and `stepconv`; Layer 1 (scanning, decoding, the writer, raw JSON) doesn't.

- **`cad-scene` is the hub:** the scene model, the tessellator interface, the tessellation driver and the export view. Importers produce scenes, backends implement its interface, exporters consume scenes; none of them knows the others.
- **Licenses:** everything is AGPL-3.0-only (versions and commits prior to 0.2.0 were Apache-2.0 except where noted) except `tess-occt`, which keeps OCCT's LGPL-2.1 (with the Open CASCADE exception) isolated in one package, and the demo models in `test_corpus/demo/` (CC BY-SA 4.0).

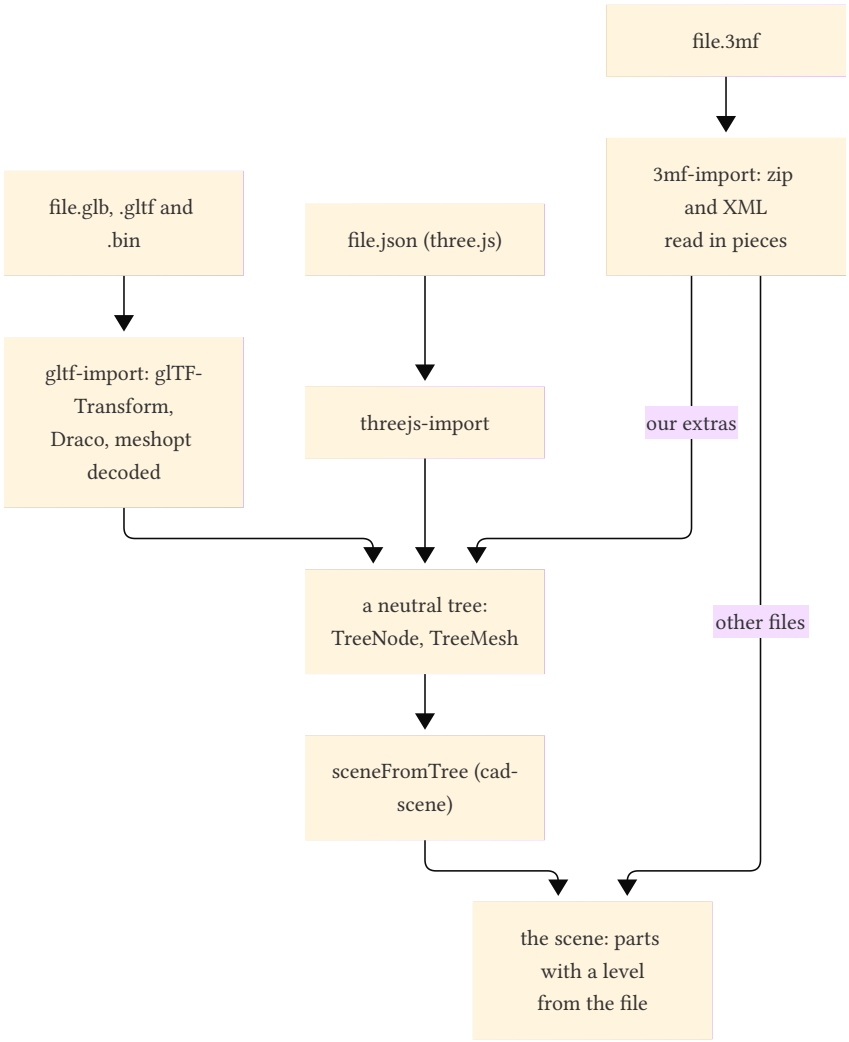
## 5.2 From a STEP File to a Mesh File




1. **Layer 1** (`step-parser`) scans the bytes once into an `InstanceIndex` (ids, byte ranges, types, references, in typed arrays) and decodes an instance only when asked for it. The writer and raw JSON work at this level.
2. **Layer 2** (`openModel`) adds the `FILE_SCHEMA`'s dictionary (and AP242's as the fallback): names, types, reverse references, value checks.
3. **Layer 3** (`extractSemantics`) reads products, occurrences and their placements, units, styles, layers and validation properties; `step2json -m semantic` writes them.

4. **`step-import`** builds the scene: components placed by occurrences, parts (one per shape representation, each with a STEP subset as its payload and an origin), materials resolved from styles, layers, overrides, names and properties.
5. **`tessellateScene`** sends one request per brep part (the deflection in the part's unit) through the cache, the worker pool and a backend (`CachedTessellator`, `WorkerTessellator`, `OcctTessellator` or `TruckTessellator`), and gets back meshes: `Float32` positions minus the part's origin, face ranges with STEP face ids, edges, curves and points. It repairs them in memory: degenerate triangles dropped, inside-out shells turned over. It also asks for each face's measures on the exact geometry (OCCT: area, perimeter, and whether the face is sound), keeps them on the part, and judges each face's triangles by them (`faceFailures`). With `outlines: true` (the viewer's choice), faces that fail to mesh (no triangles, too little or too much area, no sound area to judge by) get outlines drawn from the STEP file itself (`step-import`'s `faceOutlines`: their edges along their curves), and a part that fails altogether (an error, a timeout) a mesh of its faces with only their outlines, whatever the backend.
6. **`exportView`** computes the looks (a part and its materials), the triangle groups by material and the instance tree with transforms; the exporters write `.json`, `.glb` or `.gltf` and `.bin`, or `.3mf` (all of it is what `stepconv` runs).

### 5.3 From glTF, three.js JSON or 3MF to the Scene

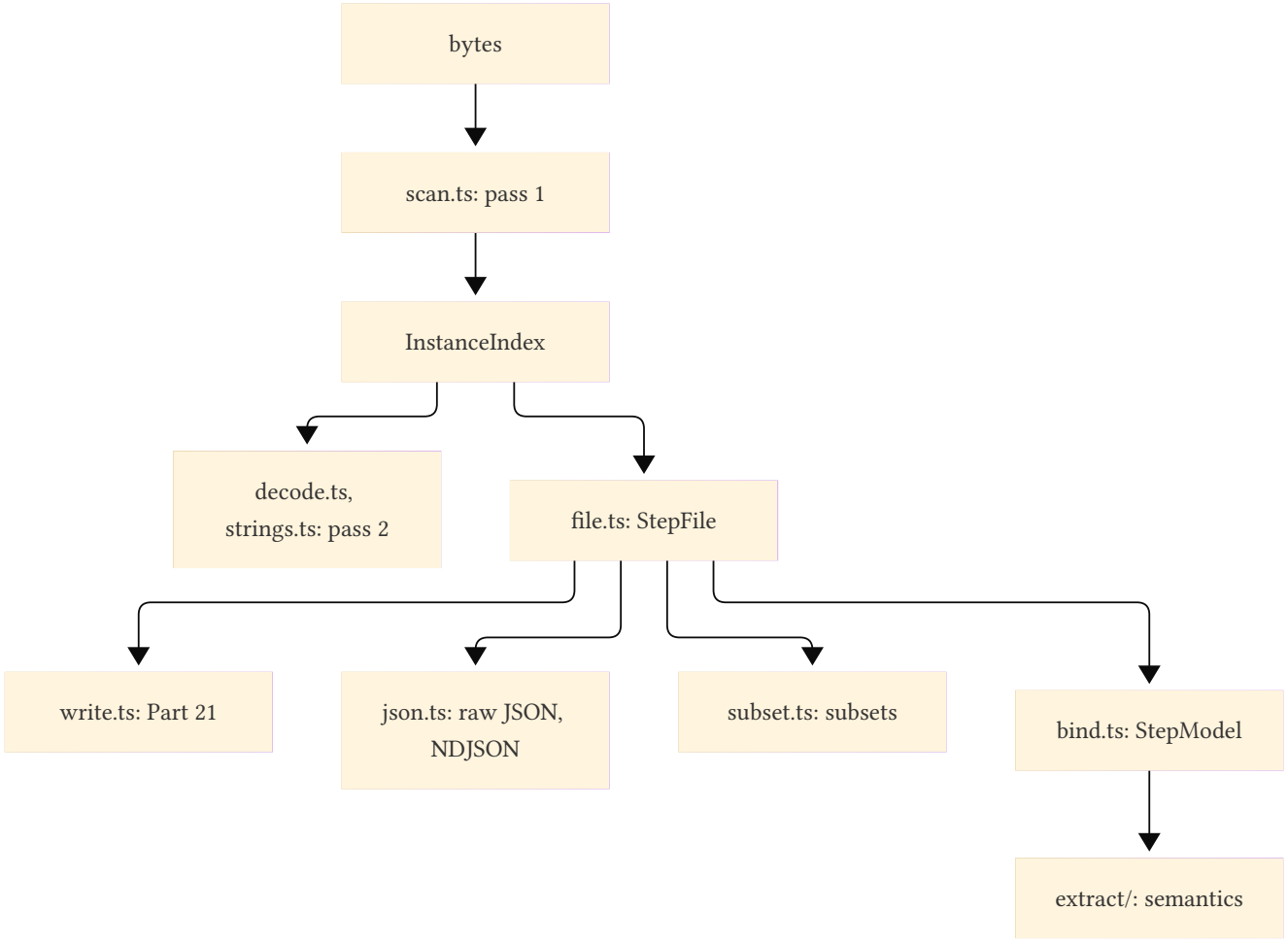



Each importer reads its format into a neutral tree of nodes, meshes and materials, and `cad-scene`'s `sceneFromTree` makes the scene from it. With our extras (`--extras`), the scene is the one the file was written from: components, parts, materials, faces by STEP id, edges, hidden placements and per-placement colours, GPU instances put back in the tree. Without them, a node holding meshes and nothing else (but its own `edges`, `curves` and `points`, as our exporters write a placed part) is a placed part, a part reference in its parent's component, its meshes merged into one part (shared where

the file reuses them), and every other node is a component, its own meshes parts; a mesh's primitives or groups are its part's faces. glTF's scene and a three.js `Scene` object hold the roots, and our exporters write no root node of their own (the turn to metres and Y up in each top node; with extras, the glTF scene's extras and the three.js `Scene`'s `userData` say what's below it), so a file read, written and read again keeps its tree, and its top nodes their frames, however many times it goes round (the user, 2026-11-01: an export's root frame is the input's, as far as the format allows: glTF and three.js keep metres and Y up, 3MF the source's unit, `Scene.sourceUnit`). Parts get their meshes as a level of detail from the file (`from: 'source'`, with the deflections it was made at where our extras say) and no B-rep, so there is nothing to tessellate, and our extras also bring back each part's unit and its faces' measures (so its faces are judged as they were) and the model's bounding sphere; the viewer, `exportView` and the exporters treat them as they treat a STEP part once it is meshed. 3MF already separates definitions (mesh and components objects) from uses (components and build items), so `3mf-import` builds the scene from them directly, a mesh object a part and a components object a component; only our own files with extras go through `sceneFromTree`, which rebuilds the tree they were written from. `corpus_checks/roundtrip_corpus.ts` takes every corpus file from STEP through each exported form and back, and finds every one drawn the same.

The mesh repairs fix what imperfect files give the backends (shells written inside out, faces that mesh to slivers), so they belong to the scene's meshes rather than to any one exporter: every consumer of `tessellateScene`'s meshes, a viewer included, gets them, each with a diagnostic, and each can be turned off.

### 5.4 step-parser

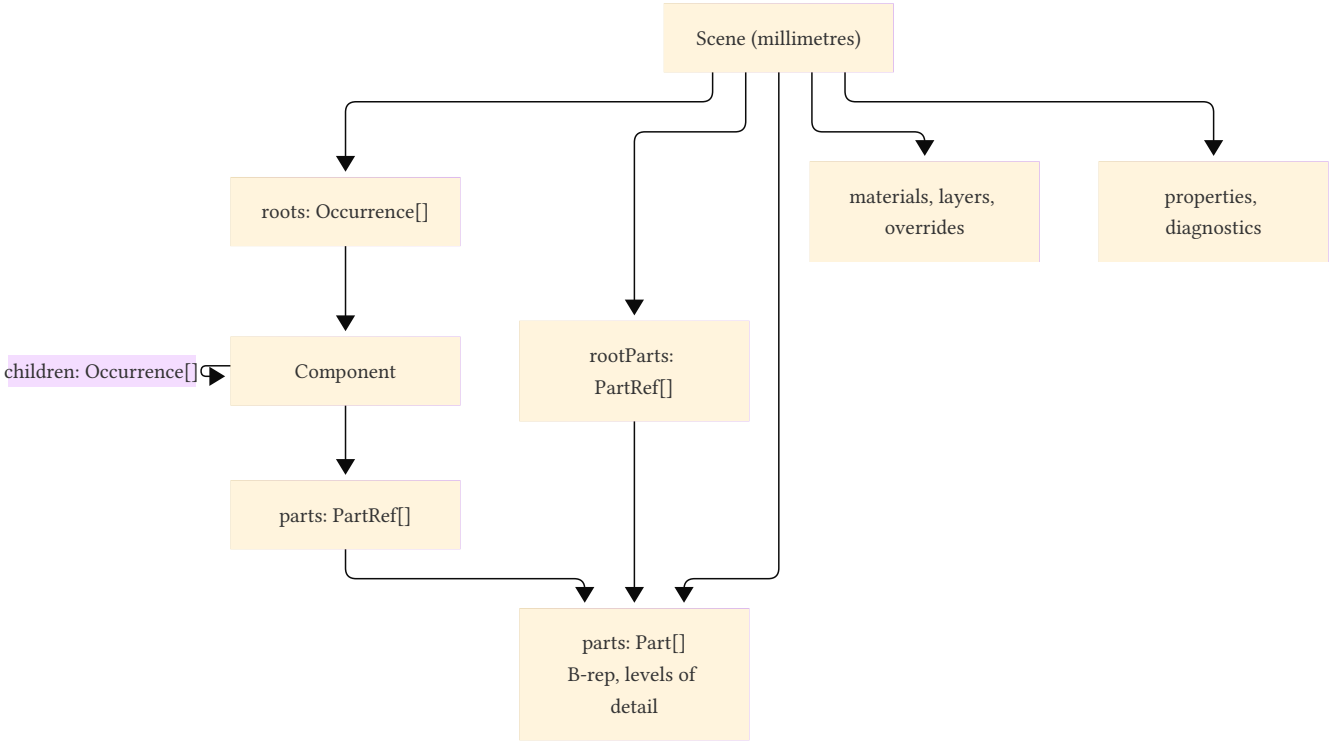



- **`scan.ts`** (pass 1, about 230 MB/s) finds the sections, the header and every instance: per instance its id, start, end and type, and all references in one `Uint32Array`; ids are looked up in a dense table or by binary search.

- **`decode.ts`, `strings.ts`** (pass 2, about 140 MB/s, on demand): an `Instance` is `{id, type, attrs: Value[]}`, a `Value` one of `int`, `real`, `str`, `enum`, `ref`, `bin`, `unset`, `derived`, `list`, `reals`, `ints`, `refs`, `typed` (lists of one kind of number, or of references, are typed arrays from 16 elements).
- **`file.ts`:** `StepFile`, with the header, schemas, data sections, `instance(id)` and `sourceBytesAt(p)`.
- **`write.ts`:** a `StepDocument` to Part 21 byte chunks (subsets copy instances' bytes as they are). **`json.ts`:** raw JSON and NDJSON, both ways (numbers through `lexical-json`). **`subset.ts`:** `closure(roots)`, `buildSubset` (bytes copied, original # ids kept), `danglingRefs`.
- **`bind.ts`:** `StepModel`, over the `step-schemas` dictionaries: `types`, `isInstanceOf`, `instancesOf`, `referrers`, and `bind(p)` for named, typed attributes with optional value checks.
- **`extract/`:** a `Reader` over the model for units, placements (ISO 10303-42), products (the tree, placements, representations, instances), styles (`effectiveStyle`) and properties, and `semanticJson`.

Every problem is a diagnostic: lexical tolerances in Layer 1 (lone apostrophes, OCCT's escape leniencies, empty parameters), unknown entities and schema violations in Layer 2 (bound by position), odd placements and units in Layer 3. Nothing fails a whole file but unreadable input.

## 5.5 Scene Model (cad-scene)



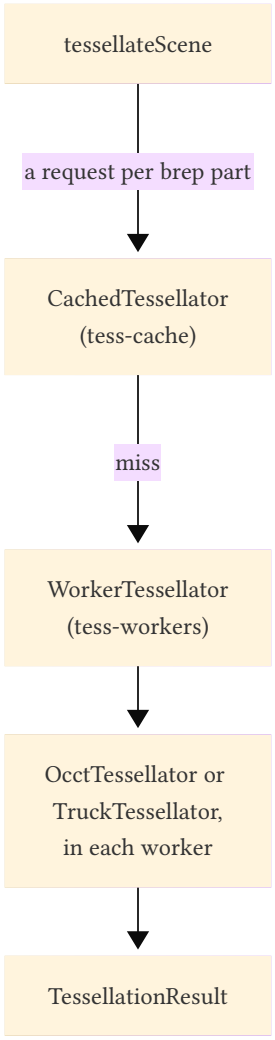

- **A component** is one product definition, stored once and placed by any number of occurrences (each with the component, a transform and a name); it has its part references, its child occurrences, and optionally an external reference, a material, a name, sources and properties.
- **A part reference** places a part: its transform, material and visibility, and where it places a leaf product, the placement's own name, sources and properties and the product's (`product`). A leaf product, a product whose only content is one shape (STEP: a product and its shape representation), is no component of its own: each placement of it is a part reference in its parent's component, or among the scene's root parts (`rootParts`), so a placed part is one node in a tree, as CAD systems and 3MF show it, not an assembly holding a part of the same name. Every importer ends with `leafParts`, which makes them so (also in files we wrote before this); `placedName` names a placed part (its own name, else its product's, else its part's).
- **A part** has exact geometry (`brep`: its format, `payload()`, a STEP subset, its root id, and `outline` for faces that fail to mesh), meshes (`lods`, levels of detail, finest first: each a mesh of triangles, edges, curves and points, from the source

file or made by a tessellator, with its deflections and maker where known, and its box), or both; it is in its own unit, with an origin near its geometry, a material and per-face materials, and (`measures`) its faces' areas and perimeters on the exact geometry, by face id, which its meshes are judged by. `tessellateScene` adds its meshes as levels; `chooseLod` picks the one the viewer and the exporters use (one at a time, for now).

- **File metadata** (`Scene.properties`): each importer's `format` and what the file says of itself: STEP's header (`name`, `timeStamp`, `authors` and `organizations` as written, `originatingSystem`, `preprocessorVersion`, `authorization`, `description`, `implementationLevel`, `schemas`), glTF's `asset`, three.js's `metadata`, 3MF's metadata by name (`unit`, `uuid`); and `author`, the author it names where the format has one (STEP's `FILE_NAME` authors, 3MF's `Designer`), leaving out names writers put where there's none (blank, `Author`, `none`, `User`, `Konstrukteur`, converters' `---` stamps: `cad-scene`'s `AUTHOR_PLACEHOLDERS`). Our files with extras give back the source's, with the file's own as `container`. Exporters write the fields each format has a place for, with or without extras (`standardMetadata`: 3MF's `Designer`, `CreationDate` from STEP's time stamp, a 3MF source's `Title`, `Description`, `Copyright`, `LicenseTerms` and `Rating`; glTF's `asset.copyright`), and with extras all of it.
- **Bounds:** each mesh has its box (`Mesh.box`, in its own coordinates, over all it draws), and the scene its bounding sphere (`Scene.bounds`, in millimetres: the file's, or estimated by the importer before meshing, or made from the meshes), for fitting a view.
- **Materials** are sRGB with opacity; **layers** name parts and faces; **overrides** change one instance's material or visibility, by its occurrence path (and a part reference's index; a root part's path is empty).
- **`instances(scene)` and `partInstances(scene)`** expand the tree: each placed part's world transform, material, per-face materials and visibility, resolved top down as the CAx-IF practice has it.

A part is one shape representation with geometry, the unit of tessellation and caching; a product spanning several representations has several part references. Lengths are millimetres; each part's geometry stays in its own unit and the transforms convert. Meshes (`Mesh`) are `Float32` positions minus the part's origin, chosen from the part's own points so that parts far from their origin keep their precision.

## 5.6 Tessellation




- **The request** (`TessellationRequest`): the payload's format and data (the subset), its root id, the origin, and the deflections in the part's unit; `tessellateScene` takes the deflections in millimetres, whether to return edges, and how many requests to keep in flight.
- **The cache** keys each result by a SHA-256 of the backend's name and version (with its module's hash), the request and the payload; its stores are memory, a directory (Node) or IndexedDB (browsers), least-recently-used within a maximum size (0 turns it off), and failures and timeouts are cached too.
- **The worker pool** runs one request per worker (`worker_threads` or Web Workers), terminates a worker whose request runs over the time limit, and replaces a worker after a trap or above 256 MiB; results' buffers are transferred, not copied.
- **The result:** positions, normals, indices, faces (index ranges with STEP face ids), edges (polylines with ids, their kinds (shared, seam, free, degenerate) and dihedral angles), and wireframe curves and points with their ids.

Each layer is itself a `Tessellator`, so they compose, and each can be used alone (a backend on the main thread, a pool without a cache).

**The OCCT backend** (`native/occt_tess.cpp`, C ABI, Emscripten): reads the subset from memory with product mode off (a bare shape representation is a root), sets the system length unit from the root's context so output is in file units, transfers, maps every resulting face back to its `ADVANCED_FACE` through the transfer bindings (faces ShapeFix split included), meshes with `BRepMesh_IncrementalMesh`, subtracts the origin in double precision, and returns faces, edges

(each with its polyline on the face triangulation, so edge points are mesh vertices), wireframe curves and points, plus OCCT's printed messages as warnings.

**The truck backend** (`native/src/lib.rs`, plain C ABI, `wasm32-unknown-unknown`): over truck unchanged, it builds the shells itself so every face and edge keeps its STEP id, meshes them, welds truck's per-corner attributes into an indexed mesh, and implements workarounds to improve results on real-world files (listed in `CLAUDE.md`). A panic costs only its part: the instance is re-instantiated.

**OCCT against truck** on the corpus. Quality, part by part at 0.05 mm (`corpus_checks/truck_occt.ts`): truck's meshes are complete and on OCCT's geometry for 2,587 of the 3,098 parts OCCT meshes (83.5%); over all parts, 2.4% of OCCT's faces are missing from truck's meshes (980 failed to mesh, 519 not converted), and some parts come back empty (geometric sets, triangulated faces, surfaces truck doesn't support); cones, surfaces of revolution and rational B-splines are where it goes wrong most. Size: its module is 326 kB brotli against OCCT's 2.18 MB. Speed, with `stepconv`'s defaults (0.1 mm, 0.5 rad, a worker pool; `corpus_checks/backend_speed.ts`), as example timings on a basic laptop (an Apple M4, 9 workers): on the 2,921 parts both mesh, truck takes 89 s of the workers' time against OCCT's 198 s (2.2 times as fast; a median of 1.7 times per part, more for larger parts), and makes 26% fewer triangles at these settings (it ignores the angular deflection). But 9 parts hang truck until the time limit (120 s when this was measured; the default is now 60 s), and one more fails after 51 s, so converting every file took 1,191 s with truck against 157 s with OCCT; without those 10 files, 60 s against 153 s. truck meshes 204 fewer parts than OCCT, and one OCCT doesn't.

### 5.7 Exporters

`exportView(scene, {edges, curves, linearDeflection, angularDeflection})` computes what every format needs once: **looks** (a part plus the material of each of its faces, so instances that look alike share one mesh), each look's triangles **grouped** by material, and the **instance tree**. Each exporter then only writes its format, in chunks, so a model's output never has to fit in one string.

A placed part's transform in the tree is the one exporters write as the mesh's node or item transform: the part reference's transform (from the part's coordinates, in its own unit, to its component's, in millimetres: so it includes the unit's scale) times a translation by the mesh's origin. The translation is there because a mesh's positions aren't stored in the part's coordinates as they are, but minus a point near the part (`Mesh.origin`), so that `Float32` keeps its precision for parts modelled far from their coordinate system's origin; adding the origin back in the transform, in double precision, puts the vertices where the part's coordinates have them.

## 6 Package APIs

All packages are ES modules with TypeScript types. Matrices are column-major `Float64Arrays` (as three.js and glTF). The examples below run inside the repository (`node example.ts`), with `assembly.stp` a STEP assembly (they were run on `test_corpus/third_party/gkv311-opencascade-dataset/step/as1-oc-214-mat.stp`, whose numbers the comments show).

### 6.1 `step-parser` (Part 21)

The parser in three layers, each usable alone: Layer 1 is the file as Part 21 (an index of every instance, instances decoded on demand, the writer, raw JSON, subsets), Layer 2 the file against its schema (names, types, attributes by name, reverse references), Layer 3 what the file means (products, assemblies and placements, units, styles, validation properties).

```
import fs from 'node:fs';
import {
  readStep, openModel, extractSemantics, productInstances, writeStep, fileDocument, writeJson, readJson, buildSubset,
} from 'step-parser';

// Layer 1: the bytes (plain or gzipped; a Uint8Array, ArrayBuffer, Blob or ReadableStream) scanned once into an
// index of every instance. Nothing is decoded yet. (parseStep(bytes) does the same for bytes already in memory.)
const file = await readStep(fs.readFileSync('assembly.stp'));
console.log(file.schemaNames, file.count);                  // [ 'AUTOMOTIVE_DESIGN { 1 0 10303 214 1 1 1 1 }' ] 6484
for (const d of file.diagnostics) console.log(d.severity, d.message);  // problems are diagnostics, never throws
```

```
// One instance decoded on demand, by its #id: {id, type, attrs}, each value tagged by its kind.
const rep = file.instance(10);
console.log(rep?.type, JSON.stringify(rep?.attrs));
// SHAPE_REPRESENTATION [{"k":"str","v":""},{"k":"refs","v":[11,15,19,23,27]},{"k":"ref","v":31}]

// Layer 2: the dictionary FILE_SCHEMA names (and AP242 for entities it doesn't know): types, attributes by name,
// subtype tests, reverse references. Instances are addressed by their position in the index, not their #id.
const model = await openModel(file);
const faces = model.instancesOf('ADVANCED_FACE');           // every instance of the entity or its subtypes
const face = model.bind(faces[0]!);                          // bind(p, {check: true}) also checks every value
console.log(faces.length, `#${face.id}`, face.get('face_geometry'));   // 53 #65 { k: 'ref', v: 80 }
console.log([...model.referrers(faces[0]!)].map((p) => `#${file.index.ids[p]}`));   // [ '#64' ] (the shell)

// Layer 3: products, the assembly tree with placements, units, styles and layers, validation properties.
const { structure, styles } = extractSemantics(model);
for (const inst of productInstances(structure).slice(0, 3)) {
  const product = structure.products[inst.product]!;
  console.log(product.partNumber, [...inst.transform.slice(12, 15)]);   // the translation, in the root's unit
}                                                            // as1 [0, 0, 0], rod-assembly [-10, 75, 60], nut ...
console.log(styles.styled.length, 'styled items,', styles.layers.length, 'layers');

// Writing: Part 21 (equivalent to the input) and raw JSON (lossless: it loads back to the same instances).
const step = writeStep(fileDocument(file));                  // or writeChunks(doc), for output in pieces
const json = writeJson(fileDocument(file));                  // or jsonChunks; NDJSON with writeJsonLines
console.log(step.length, json.length, readJson(json).data[0]!.instances.length);   // 407839 642989 6484

// A subset: one shape representation and everything it references, original #ids kept, as a file of its own.
const shape = [...structure.representations.keys()][0]!;
const subset = buildSubset(file, [file.index.position(shape)]);
console.log(`#${shape}`, subset.positions.length, subset.bytes.length);   // #3812 2392 168061
```

Also: `file.instanceAt(p)` and `file.sourceBytesAt(p)` (an instance by position, or its bytes as written), `file.header` and `file.dataSections`; `model.types(p)`, `model.isInstanceOf(p, entity)`, `model.resolve(id)` (an id to its position); `makeHeader({...})` for a new file's header; `closure(file, roots)` and `danglingRefs(file, positions)` (what a subset would hold, and its references to instances the file doesn't define); `effectiveStyle` and `semanticJson` (what `step2json -m semantic` writes). The raw model (`model.ts`): `Instance` is `{id, type, attrs: Value[]}` or, for a complex instance, `{id, parts: [{type, attrs}]}`; `Value` is one of `int` (number or bigint), `real`, `str`, `enum`, `ref`, `bin`, `unset`, `derived`, `list`, `typed`, and the homogeneous lists `reals`, `ints`, `refs` (canonical: `makeList` picks the kind).

## 6.2 `step-schemas`

`Schema` dictionaries for AP203 (editions 1 and 2), AP214 (edition 3) and AP242, generated from the EXPRESS long forms in stepcode and committed as JSON (`dict/*.json`, 1.1 MB, 143 kB gzipped; see Package Dependencies for when they are needed). Each holds the entities with their attribute slots in Part 21 order (supertypes first, redeclarations and derived attributes applied), the defined types, selects and enumerations: enough to bind a Part 21 instance's values to names and check their types, not the schemas' rules (WHERE clauses aren't checked).

```
import { DICTIONARIES, dictionaryForFileSchema, loadSchema } from 'step-schemas';

console.log(DICTIONARIES.map((d) => d.id));                  // [ 'ap203', 'ap203e2', 'ap214e3', 'ap242' ]
// A FILE_SCHEMA entry to a dictionary, by the schema's name (case and the object identifier ignored).
const id = dictionaryForFileSchema(['AUTOMOTIVE_DESIGN { 1 0 10303 214 1 1 1 1 }'])!;   // 'ap214e3'
const ap214 = await loadSchema(id);                           // a dynamic JSON import: only what's used is loaded

// An entity's attribute slots in Part 21 order, inherited ones first; `derived` slots hold '*' in Part 21.
console.log(ap214.instanceAttributes('ADVANCED_FACE').map((a) => a.name));
// [ 'name', 'bounds', 'face_geometry', 'same_sense' ]
console.log(ap214.ancestors('ADVANCED_FACE'));               // the supertypes from the top down, then itself
// [ 'REPRESENTATION_ITEM', 'TOPOLOGICAL_REPRESENTATION_ITEM', 'FACE', 'GEOMETRIC_REPRESENTATION_ITEM',
//   'FACE_SURFACE', 'ADVANCED_FACE' ]
console.log(ap214.isSubtypeOf('ADVANCED_FACE', 'FACE'), ap214.subtypes('FACE_SURFACE'));   // true [ 'ADVANCED_FACE' ]
console.log([...ap214.enumValues('SI_PREFIX')].slice(0, 4));  // [ 'EXA', 'PETA', 'TERA', 'GIGA' ]
// A complex instance's partial: the slots NAMED_UNIT contributes to (LENGTH_UNIT()NAMED_UNIT(*)SI_UNIT(...)).
```

```
console.log(ap214.partialAttributes('NAMED_UNIT', ['LENGTH_UNIT', 'NAMED_UNIT', 'SI_UNIT']).map((a) => a.name));
// [ 'dimensions' ]
```

Also `selectLeaves(type)` (the entities and types a SELECT can hold, through nested selects), `entity(name)` and `type(name)` (the raw definitions) and `loadDictionary(id)` (the JSON itself). `npm run gen -w step-schemas` regenerates the dictionaries from `third_party/stepcode`, and a test checks that the committed ones are what the generator makes.

### 6.3 `lexical-json`

JSON that keeps integers and reals apart by how they are written, as Part 21 does (`5` and `5.0` differ there, and a schema doesn't always say which a value is). Format-agnostic, for raw JSON of other CAD formats later.

```
import { parseJson, stringify, formatReal } from 'lexical-json';

// Integers come back as bigint, reals as number, as they were written: 5 and 5.0 stay different.
const v = parseJson('{"count":5,"length":5.0,"big":12345678901234567890,"tiny":1e-7}') as Record<string, unknown>;
console.log(typeof v.count, typeof v.length, v.big);         // bigint number 12345678901234567890n
// And back: bigints as integers, numbers always as reals (shortest round-trip text, with '.0' or an exponent),
// typed arrays as lists.
console.log(stringify({ count: 5n, length: 5, list: new Float64Array([0.1, 2]) }));
// {"count":5,"length":5.0,"list":[0.1,2.0]}
console.log(formatReal(1e21), formatReal(-0));               // 1e+21 -0.0
```

`parseJson` takes bytes or a string; `formatInt` and `formatString` write single values.

### 6.4 `cad-scene`

The scene model every importer produces and every exporter consumes (see Scene Model), the `Tessellator` interface backends implement, `tessellateScene` to drive one over a scene, and `exportView` for exporters (see Exporters). The model's types: `Scene`, `Component`, `Occurrence`, `PartRef`, `Part` (with `Brep`, `Lod` and `Measures`), `Mesh`, `Box`, `Sphere`, `SceneBounds`, `Material`, `Layer`, `InstanceOverride`, `Diagnostic`, `SourceId`; the interface's: `Tessellator`, `TessellatorInfo`, `TessellationRequest`, `TessellationResult`.

```
import fs from 'node:fs';
import { importStep } from 'step-import';
import { instances, partInstances, tessellateScene, validate } from 'cad-scene';
import { OcctTessellator } from 'tess-occt';

const { scene } = await importStep(fs.readFileSync('assembly.stp'));
console.log(scene.components.length, scene.parts.length, scene.materials.length);   // 9 5 5
console.log(validate(scene));                                  // [] (indices in range, transforms finite, ...)

// Components are stored once and placed by occurrences; instances() expands the tree, with world transforms (mm).
for (const c of instances(scene)) console.log('  '.repeat(c.path.length) + scene.components[c.component]!.name);
//   as1
//     rod-assembly
//       nut ...

// Every placed part: its transform from its own coordinates and unit to the scene's millimetres, its material and
// per-face materials as the styles resolve for that instance, and whether it's visible.
const placed = [...partInstances(scene)];
const first = placed[0]!;
const part = scene.parts[first.part]!;
console.log(placed.length, part.name, part.brep?.format, part.unit, scene.materials[first.material!]);
// 18 nut step 1 { color: [ 1, 0, 0 ], opacity: 1, sources: [] }

// Tessellation: one request per part with B-rep the backend takes (its STEP subset), up to `concurrency` at once
// (set it to a worker pool's size); here the backend runs on this thread. Each mesh becomes a level of its part.
const occt = new OcctTessellator();
await occt.init();
const { meshes, diagnostics } = await tessellateScene(scene, occt, {
  linearDeflection: 0.1,            // millimetres, converted to each part's unit
  angularDeflection: 0.5,           // radians
  edges: true,                      // B-rep edges as polylines too, with their kinds and dihedral angles
```

```
  onPart: (part, mesh, done, total) => {},   // progress
});
occt.dispose();
const mesh = part.lods[0]!.mesh;  // positions minus mesh.origin, in the part's unit; faces by STEP face id
console.log(mesh.positions.length / 3, mesh.indices.length / 3, mesh.faces.length, mesh.faces[0]!.source,
  mesh.origin, diagnostics.length);  // 168 156 8 65 [ 5, 10, 1.5 ] 0
```

The deflections come from a meshing profile (`MESH_PROFILES` in `cad-scene`, the one place they're defined): `profile: 'coarse'` (0.2 mm, 30 degrees), `'medium'` (0.1 mm, 15 degrees; `DEFAULT_MESH_PROFILE`, used when nothing is given) or `'fine'` (0.05 mm, 7.5 degrees); `linearDeflection` and `angularDeflection`, as in the example, replace the profile's values, one or both (`meshDeflections(opts)` gives the result, `meshProfileOf(deflections)` the profile they match, if any). Over the corpus with OCCT, against 0.1 mm and 28.6 degrees (the default before 2026-10-26): coarse has 0.59 times the triangles, medium 1.25 times (the median part 1.7 times), fine 3.3 times, and fine's tight angle makes 14 parts time out at 60 s.

`tessellateScene` repairs what it can in every mesh, with a diagnostic each time, so every consumer of the scene gets the repaired mesh: triangles with two corners at one position are dropped, and closed shells that are inside out and enclosed by nothing else (so can't be voids) are turned over. `orient: false` and `dropDegenerate: false` keep the backend's meshes as they are, and `parts` limits it to some parts. Also: `identity()`, `multiply(a, b)`, `scaling(s)` for matrices, and `Y_UP_METRES` (millimetres and Z up to metres and Y up, the glTF and three.js exporters' root); `sha256(bytes)` (Web Crypto's, or our own `sha256Js` where Web Crypto is missing, as in pages that aren't a secure context); `weld(positions)` (vertices merged by exact position), `manifoldProblems(positions, triangles)`, `orientShells(mesh)` and `dropDegenerate(mesh)`, the mesh helpers behind the repairs and the 3MF exporter; `addLod(part, lod)` and `chooseLod(part, {linearDeflection, angularDeflection})` (a part's levels: added finest first, and the one used: ours at those settings, else the nearest not coarser, else the finest); `TessellationError`, which a backend throws for a failure that is the part's, not the host's.

**Faces that fail to mesh.** `tessellateScene` (`measures`, default true) asks the backend for each face's measures where the part has none, and keeps them as `Part.measures` (`by`, who measured; `faces`, by face source id: `area`, `perimeter`, in the part's units, and `valid`, whether the backend found the face sound). `faceFailures(mesh, { expected, measured: { measures, deflection } })` judges each face: `failed` (no triangles, for a face with something to mesh at that deflection), `partly` (its triangles short of its area), `excess` (well over it) or `unverified` (no sound area to judge by). With M the mesh's area, A the face's, d the deflection and P its perimeter, a face is partly when A - M > max(3 % of A, d x P) and excess when M - A > max(10 % of A, d x P) (`MISSING_FRACTION`, `EXCESS_FRACTION`, `BOUNDARY_SLACK`): the fractions are relative tolerances, and d x P allows the strip along its boundary that a legitimate mesh, up to d inside the surface, may lose. Without measures, a face fails only with no triangles. Over the corpus the median face's mesh is 0.9994 of its area; OCCT measures every face for about 3 % of its meshing time.

**Bounding volumes.** Every mesh gets its box when it's made (`meshBox`, `Mesh.box`: the importers through `boundScene`, `tessellateScene` for its own), never read from a file (every importer holds the whole mesh, and a file's box can disagree with it: codecs' quantization, writers' errors); exporters write it where the format has a slot (glTF's accessors' `min` and `max`) and else in our extras (`box` on three.js and 3MF part nodes). The scene's sphere (`Scene.bounds`, `sceneBounds`) is our extras' where the file has them (for the whole scene: glTF's scene's extras, three.js's `Scene`'s `userData`, 3MF's `cadio:tree`), else estimated by the importer from each placed part's mesh box, or for a B-rep part the box of its vertices (STEP's `VERTEX_POINT`s, `Brep.vertexBox`: no bound, as curved faces bulge past their vertices), each box's 8 corners placed in the world, the centre the middle of their box and the radius the farthest corner (`boxesSphere`). `tessellateScene` makes it again from the meshes alone when it's done, which can shrink an estimate. On six models (as1, the bicycle, `bug24383_224013.stp`, ohmm, rpbp, lightarm) the estimate's radius was within 0-5 % of the meshes', and the meshes' within 0-3 % of the farthest vertex from the box's centre on as1, rpbp and ohmm, 13 % on the bicycle (its turned parts' boxes reach further). Zoom selected makes the selection's sphere the same way.

## 6.5 `step-import`

STEP to a scene. `importStep(input)` reads, binds and extracts (the three layers of `step-parser`), then builds the scene; `sceneFromStep(model, semantics)` does the last step for a model already opened. It returns the scene with the model and semantics it came from, and `partOf`, the part index of each shape representation id. `onProgress` is told each phase as it begins (`reading`, `structure`, `parts`, `assembly`), and through `parts` how many of the file's shape representations it has looked at (`done` of `total`): most of a large file's time (LMP1's 271 MB: reading 0.9 s, structure 1 s, parts 12.5 s, assembly under 0.1 s in Node, Apple M4).

```
import fs from 'node:fs';
import { importStep } from 'step-import';

const { scene, model, semantics, partOf } = await importStep(fs.readFileSync('assembly.stp'));
for (const d of scene.diagnostics.slice(0, 5)) console.log(d.severity, d.message, d.source?.id);
console.log(partOf.size, 'representations as parts,', semantics.structure.products.length, 'products');   // 5, 9
for (const c of scene.components) console.log(c.name, c.properties);   // { partNumber: 'nut', lifeCycleStage: ... }
```

What it does with the file:

- **Components** from product definitions, with their name, part number, version and description as properties; **occurrences** from the assembly's usage occurrences (and mapped items), with their placements converted to millimetres. Writers' known placement errors are corrected as OCCT does (swapped representations, swapped items), with Alibre's exception.
- **Parts** from shape representations with geometry (B-rep solids, shells, surface models, wireframes, tessellated geometry); annotations and 2D drawings are skipped with a note. Each part's payload is a STEP subset built on demand, its origin the centre of its vertices' bounding box (of all its points when it has no vertices: control points can lie far off), its unit its representation's.
- **Tessellated geometry** (AP242's, as the CAx-IF's practice describes it): a representation holding only tessellated items is read as it is, into a level of detail from the file (`from: 'source'`) with no B-rep, so nothing is meshed: tessellated solids, shells and faces (triangle strips and fans included), triangulated surface sets (a face each), with every face's STEP id; the file's normals, else computed per face; edges from the file's tessellated edges, else each face's boundary, shared where two faces meet; curve sets, wires and edges as curves, vertices and point sets as points; geometric sets with repositioned items, placed; `closed` for solids and for shells linked to a closed or open shell; AP242 edition 2's accuracy parameters as the level's deflections. A product's tessellation of its B-rep becomes a level of the B-rep's part (paired by its links to the B-rep, else with the product's one B-rep part with faces), its linked faces taking their B-rep face's id: shown at once, replaced by the backend's mesh when meshed, kept if that fails. Curved (`CUBIC_BEZIER_...`) triangles and other items it doesn't read leave their representation to the backend, with a note; `importStep(bytes, { tessellated: 'backend' })` (`stepconv --tessellated-by-backend`) sends every tessellated representation to the backend, as before. OCCT 8.1 reads strips and fans wound the other way, so its meshes of strip-built solids come back inside out (and are turned over by `tessellateScene`, with a warning).
- **Materials** from styles: per part when every face resolves to one colour, else per face; boundary colours when nothing else colours a face; context-dependent overrides per instance; invisibility.
- **Layers**, external references (components whose geometry is in another file).
- **Names**: generic part names (`NONE`, `#123`, blank) take their product's; blank product names their part number.
- **Diagnostics**: parts with references to instances the file doesn't define get a warning naming them; representations without a length unit are read as millimetres, with a warning.

## 6.6 `threejs-import`

three.js JSON (the Object Scene format, as `Object3D.toJSON` and `export-threejs` write it) to a scene. `importThreejs(input)` takes the text, its bytes or the parsed object, and returns the scene and its diagnostics (problems in the file are diagnostics, never throws).

```
import fs from 'node:fs';
import { exportView, viewDifferences } from 'cad-scene';
```

```
import { threejsJson } from 'export-threejs';
import { importThreejs } from 'threejs-import';

const { scene, diagnostics } = importThreejs(fs.readFileSync('assembly.json'));
// Our own files read back as written: export, import, and nothing drawn differently.
const back = importThreejs(threejsJson(scene, exportView(scene), { extras: true })).scene;
console.log(viewDifferences({ scene, view: exportView(scene) }, { scene: back, view: exportView(back) }));  // []
```

- **Our files with extras** (`--extras`: the `Scene`'s `userData` says millimetres, Z up below each top object's turn; before 2026-11-01, a root group's) come back as the scene they were written from: components by their index, with their names, sources and properties, each placed by its occurrences; parts by theirs, each with one level of detail from the file (`from: 'source'`) holding its faces by STEP face id, its B-rep edges, curves and points, and `closed`; materials by their scene index; and the hidden placements and per-placement colours as overrides (one per placement where it differs from the part's first: `InstanceOverride.ref`). What `exportView` left out (empty nodes) stays out.
- **Other files** keep their own tree: an object holding meshes and nothing else (but its own `edges`, `curves` and `points`) a placed part, its meshes one part, and every other object a component placed once (its name and matrix), its own meshes parts; a part shared wherever the file reuses its geometry and materials, its geometry's groups (if more than one) its faces, numbered from 0, each with its material (a mesh of one group has no face ids); an instanced mesh a placement per instance; lines (segments, strips, loops) and points parts of curves and points, materials as their colour and opacity; the root from metres, Y up, to millimetres, Z up. A child named `edges` of only lines beside one mesh, as our exporters write B-rep edges without extras, is that mesh's edges. Our files with extras written before 2026-10-24 (millimetres, no up axis, no root matrix) are read in millimetres as they are, otherwise as other files are (their extras aren't read). Geometries other than `BufferGeometry` (made from parameters) are skipped, and textures ignored, each with a warning.
- **`cad-scene`'s `sceneFromTree`** does the work from a neutral tree of nodes and meshes (`TreeNode`, `TreeMesh`, `TreeMaterial`), which `gltf-import` uses too; `viewDifferences(a, b, {tolerance, skipHidden})` compares what two scenes' export views draw, placement by placement (paired by name along the path and the centre of what each draws): names, visibility, each triangle's corners in the world with its face id and colour, edges, curves and points, each matched to one within the tolerance.

### 6.7 `gltf-import`

glTF 2.0 to a scene, read with glTF-Transform: `await importGltf(bytes)` for a GLB, `await importGltf({ json, resources })` for a `.gltf` and the files its buffers name (data URIs need none; one not given is an error). Draco and meshopt are decoded on reading (their decoders imported only for files that use them); in a bundled page, `importGltf(input, { draco: { locateFile: () => wasmUrl } })` tells Draco's decoder where its WebAssembly is (or `wasmBinary`, its bytes), as the viewer's import worker does. It returns the scene and its diagnostics, as `threejs-import` does, through the same `sceneFromTree`.

```
import fs from 'node:fs';
import { exportView, viewDifferences } from 'cad-scene';
import { glb } from 'export-gltf';
import { importGltf } from 'gltf-import';

const { scene, diagnostics } = await importGltf(fs.readFileSync('assembly.glb'));
// Our own GLBs read back as written, within the codec's quantization.
const { bytes } = await glb(scene, exportView(scene), { extras: true, hidden: 'extension' });
const back = (await importGltf(bytes)).scene;
console.log(viewDifferences({ scene, view: exportView(scene) }, { scene: back, view: exportView(back) },
  { tolerance: 0.05 }));   // []
```

- **Our files with extras**, beyond what three.js JSON's give: faces by Draco's `_FACE` attribute (Draco reorders triangles); hidden placements written with `KHR_node_visibility` hidden, and those left out (the default) back as hidden where another instance of their component has them, else absent (compare with `viewDifferences`' `skipHidden`); GPU instances (`EXT_mesh_gpu_instancing`) put back in the tree where their extras' `path` says, with their edges, curves

and points, the tree nodes the export left empty made again from the names along the path (tree nodes carry their `path` in their extras since 2026-10-29).
- **Other files**, as three.js JSON's: a node of meshes alone a placed part, every other node a component, the scene's nodes the roots, a part shared wherever the file reuses it, its triangle primitives (if more than one) its faces, numbered from 0, each with its material (Onshape writes a primitive per face, so its parts close up for section caps); line strips and loops as segments, triangle strips and fans as triangles; base colour factors converted from linear (glTF's) to sRGB, except Onshape's (`srgbFactorsFrom(generator)`), which are sRGB already; GPU instances a placement per instance; textures ignored (a warning). glTF-Transform's own messages (an optional extension it doesn't know) become diagnostics.

### 6.8 `3mf-import`

3MF to a scene: the core specification, the Materials extension's colours (`basematerials`, `colorgroup`, per-triangle properties) and the Production extension's model parts (objects in other `.model` parts of the package, by `p:path`). `import3mf(bytes, { creaseAngle })` returns the scene and its diagnostics, as the other importers do. The package is read by our own zip reader (fflate's streaming inflater) and XML tokenizer, the model part in 1 MB pieces, so a model of hundreds of megabytes of XML (an Onshape export of 377 MB, 4.4 M triangles: about 6 s on an Apple M4) is never held as one string; nothing else is needed, and it runs in a worker (no `DOMParser`).

```
import fs from 'node:fs';
import { exportView, viewDifferences } from 'cad-scene';
import { threemf } from 'export-3mf';
import { import3mf } from '3mf-import';

const { scene, diagnostics } = import3mf(fs.readFileSync('assembly.3mf'));
// Our own 3MF with extras read back as written (3MF has no lines; hidden placements kept): no differences, but
// for triangles with two corners at one position, which the exporter drops (cnc-box's 4 of 96,566).
const r = threemf(scene, exportView(scene), { extras: true, hidden: 'keep' });
const back = import3mf(Buffer.concat(r.chunks)).scene;
console.log(viewDifferences({ scene, view: exportView(scene) }, { scene: back, view: exportView(back) },
  { lines: false }));   // [] for lightarm
```

- **Every file:** a mesh object is a part (`closed` true for `type="model"`, which 3MF requires to be a closed manifold, false for `surface`, unknown otherwise), with one level of detail from the file; a components object a component, placed by its components; the build's items the roots (a components object's a root occurrence, a mesh object's a root part, the item's part number, UUID and metadata its properties); a components object of one mesh object and nothing else is that part placed (`leafParts`). Faces: all of a mesh object's triangles of one colour are one face, numbered from 0, with that colour as its material. Normals are computed, smooth across edges where the triangles' normals differ by less than the crease angle (default 30 degrees, `DEFAULT_CREASE_ANGLE`) and split above it. Units are converted to millimetres; Z stays up. An object's or item's type, part number, Production UUID and metadata become its properties. Textures, beam lattices, slices, displacement and volumetric data are skipped with a warning, as is anything in an unknown namespace; a `requiredextensions` naming one we don't read is warned of and the file read anyway.
- **Our files with extras** (`stepconv --extras`): the tree, names, components and their sharing, per-placement colours and hidden placements kept (`--keep-hidden`) come back as the scene was, through `sceneFromTree` (from the `cadio:tree` and `cadio:materials` metadata), and faces by their STEP ids (each object's `cadio:faces` metadata); edges, curves and points don't (3MF has no lines), nor hidden placements left out. With `--3mf-assembly` the components are read as written, with their face ids (but not the parts' measures nor the model's sphere, which only the default form's `cadio:tree` carries). Our files without extras are read as anyone's: with the default build, a flat list of placed parts.

### 6.9 `tess-occt`, `tess-truck`

The two backends, each a `Tessellator`: give it one part's STEP subset, get back its triangles with the STEP face each came from.

```
import fs from 'node:fs';
import { buildSubset, parseStep } from 'step-parser';
import { OcctTessellator } from 'tess-occt';
import { TruckTessellator } from 'tess-truck';

const file = parseStep(fs.readFileSync('assembly.stp'));
const data = buildSubset(file, [file.index.position(3812)]).bytes;   // a shape representation and its closure
for (const tess of [new OcctTessellator(), new TruckTessellator()]) {
  await tess.init();                // loads and compiles the module: from the file system in Node, by fetch
elsewhere
  console.log(tess.info);           // name, version (with the module's hash), license, what it supports
  const r = await tess.tessellate({ format: 'step', data, rootId: 3812, linearDeflection: 0.1,
    angularDeflection: 0.5, edges: true, origin: [0, 0, 0] });   // deflection and origin in the file's units
  console.log(r.positions.length / 3, r.indices.length / 3, r.faces.length, r.edges?.length, r.diagnostics);
  console.log(tess.memoryBytes?.());  // the WebAssembly heap, which never shrinks
  tess.dispose();
}
```

OCCT returns face and edge ids, edges, wireframe curves and points; truck returns face and edge ids and edges (no curves), and ignores the angular deflection.

**A host's own build.** By default each backend loads the module beside its package (`dist/`). A host that already loads WebAssembly of its own can hand a backend its build instead (a sketch: `compiledModule` and `glueUrl` are the host's):

```
// OCCT: the Emscripten glue (its factory, or the .mjs's URL) and the module (bytes, URL or compiled), from one build.
await occt.init({ glue: 'https://example.com/occt/host_occt.mjs', wasm: compiledModule,
  version: 'host-occt 2026.10' });
// truck: the module (bytes, URL or compiled).
await truck.init({ wasm: 'https://example.com/truck/tess_truck.wasm' });
// In a worker pool, the same as the backend's options (URLs or a compiled module; a function can't be posted).
new WorkerTessellator({ backend: { module: import.meta.resolve('tess-occt'), export: 'OcctTessellator',
  options: { glue: glueUrl, wasm: compiledModule, version: 'host-occt 2026.10' } } });
```

- **An OCCT build must link our wrapper**, `native/occt_tess.cpp`, with the exports and settings `third_party/README.md` lists (OCCT 8, `-fwasm-exceptions`, `-UOCC_CONVERT_SIGNALS`, a modular ES-module build, the wrapper's functions exported). An OCCT module without it (`occt-import-js`, `opencascade.js`) can't be used: the backend calls the wrapper, not OCCT's classes. `init` checks the exports and fails naming what's missing. truck's module is likewise our wrapper (`native/`) and truck built as one.
- **The cache key:** `info.version`, which `tess-cache` keys include, names the module by a hash of its bytes, so a cache never answers from another build's results. A compiled `WebAssembly.Module` has no bytes to hash, so `version` is then required, and must change whenever the module does; with bytes, `version` replaces the default name and the hash is kept.
- **What's shared** is the download and the compilation, not a running instance: each tessellator (each worker's, in a pool) instantiates the module with its own memory.

### 6.10 `tess-workers`

Any backend in a pool of workers: `worker_threads` in Node, module Web Workers in browsers (the package's export conditions choose). Each worker loads the backend from `backend.module` (a URL or resolvable specifier) and `backend.export` (the class name), with `backend.options` passed to its `init`. A request that takes longer than `timeoutMs` terminates its worker and fails with a `TessellationError`; the part fails alone, and a fresh worker takes its place. Workers are also replaced after a backend trap, after `maxPartsPerWorker` parts, or when the backend's memory grows beyond `maxMemoryBytes` (WebAssembly memory never shrinks). Defaults: cores - 1 workers, 60 s timeout, recycling above 256 MiB, no limit on parts.

```
import fs from 'node:fs';
import { tessellateScene } from 'cad-scene';
import { importStep } from 'step-import';
import { WorkerTessellator } from 'tess-workers';
```

```
const { scene } = await importStep(fs.readFileSync('assembly.stp'));
const pool = new WorkerTessellator({
  backend: { module: import.meta.resolve('tess-occt'), export: 'OcctTessellator' },
  workers: 4, timeoutMs: 30_000, maxMemoryBytes: 512 * 2 ** 20,
});
await pool.init();                    // starts the workers, each loading the backend
const { meshes } = await tessellateScene(scene, pool,
  { linearDeflection: 0.1, angularDeflection: 0.5, concurrency: 4 });   // as many in flight as there are workers
pool.dispose();                       // terminates the workers
```

In a browser, the backend's module is fetched by each worker, and the pool's results' buffers are transferred, not copied. A bundled application (the viewer, with Vite) can't have its workers import a backend by a URL known only at run time: it gives the pool its own worker entry instead, which imports the backends statically and passes them to `serveWebWorker` (from `tess-workers/web-worker`), and spawns it with `webWorkerHandle(new Worker(new URL('./tess-worker.ts', import.meta.url), { type: 'module' }))` and `PoolTessellator` (from `tess-workers/browser`); `backend.module` then names one of the modules it passed (see `packages/viewer/src/tess-worker.ts`).

### 6.11 `tess-cache`

A `Tessellator` in front of another that keeps its results, keyed by a SHA-256 of the backend's name and version (which includes its module's hash), the request and the payload: a part asked for again, with the same deflections, is answered from the store. Stores: `MemoryStore(maxBytes)`, `FileStore({dir, maxBytes})` (Node; a directory, so it survives the process) and `IndexedDbStore({name, maxBytes})` (browsers; `tess-cache/browser` names the browser entry for type checking under Node), each least-recently-used within its maximum size, 0 disabling it.

```
import fs from 'node:fs';
import os from 'node:os';
import path from 'node:path';
import { tessellateScene } from 'cad-scene';
import { importStep } from 'step-import';
import { CachedTessellator, FileStore } from 'tess-cache';
import { WorkerTessellator } from 'tess-workers';

const { scene } = await importStep(fs.readFileSync('assembly.stp'));
const pool = new WorkerTessellator(
  { backend: { module: import.meta.resolve('tess-occt'), export: 'OcctTessellator' } });
const dir = fs.mkdtempSync(path.join(os.tmpdir(), 'tess-cache-'));
const tess = new CachedTessellator(pool, new FileStore({ dir, maxBytes: 256 * 2 ** 20 }));
await tess.init();
for (const run of [1, 2]) {
  await tessellateScene(scene, tess, { linearDeflection: 0.1, angularDeflection: 0.5, concurrency: 4 });
  console.log(tess.stats());          // run 1: { hits: 0, failureHits: 0, misses: 5, errors: 0 }; run 2: 5 hits
}
tess.dispose();
```

Failures are cached too (results with an error, and timeouts: a cached timeout answers only for a time limit no longer than the one that ran out); `retryFailures: true` sends them to the backend again. The first time a cache sees a backend version, it deletes that backend's entries for other versions, which would never be read again (`keepOtherVersions: true` keeps them); other backends' entries are kept. A store's failures never fail a request: the backend answers instead, and `stats().errors` counts them. A miss costs nothing measurable, a hit about 0.4 ms.

### 6.12 `export-threejs`, `export-gltf`, `export-3mf`

Each takes the scene and its export view, and writes one format (see what each format holds, under `stepconv` in Command-Line Tools). The output is deterministic: the same scene gives the same bytes.

```
import fs from 'node:fs';
import { Readable } from 'node:stream';
import { pipeline } from 'node:stream/promises';
import { exportView, tessellateScene } from 'cad-scene';
import { threemf } from 'export-3mf';
```

```
import { glb, gltfDocument } from 'export-gltf';
import { threejsChunks } from 'export-threejs';
import { importStep } from 'step-import';
import { OcctTessellator } from 'tess-occt';

const { scene } = await importStep(fs.readFileSync('assembly.stp'));
const occt = new OcctTessellator();
await occt.init();
const meshing = { linearDeflection: 0.1, angularDeflection: 0.5 };
await tessellateScene(scene, occt, { ...meshing, edges: true });
occt.dispose();

// The export view, computed once for any number of exporters (each part's level at these settings); edges and
// curves only if asked for.
const view = exportView(scene, { ...meshing, edges: true });
console.log(view.looks.length, 'looks');                    // 5

// three.js JSON in chunks, so a large model never has to be one string; THREE.ObjectLoader reads the result.
await pipeline(Readable.from(threejsChunks(scene, view, { name: 'as1', edgeColor: [0.2, 0.2, 0.2] })),
  fs.createWriteStream('assembly.json'));               // or threejsJson(scene, view, opts) for one string

// GLB (metres and Y up, as glTF has it), Draco-compressed, with hidden instances kept as KHR_node_visibility; or the
// glTF-Transform document, to process further (merge, ...) before writing it with glTF-Transform's own tools (which
// compress it with the encoder registered: `draco3d.encoder` from draco3dgltf, `meshopt.encoder` from meshoptimizer).
const { bytes, diagnostics } = await glb(scene, view, { hidden: 'extension', unlitLines: true });
fs.writeFileSync('assembly.glb', bytes);
const { document } = await gltfDocument(scene, view, { compression: 'none', extras: true });
console.log(document.getRoot().listMeshes().length, 'glTF meshes');   // 10 (5 of faces, 5 of edges)

// 3MF: closed meshes only (or open ones as surface objects), an item per placed part or the assembly as components.
const r = threemf(scene, view, { build: 'assembly', colors: 'colorgroup', nonManifold: 'surface' });
await pipeline(Readable.from(r.chunks), fs.createWriteStream('assembly.3mf'));
```

Options:

- **three.js** (`threejsChunks`, `threejsJson`): `name` (the root object's), `doubleSided`, `defaultColor` (faces without a material; sRGB), `edgeColor`, `edgeKinds` (which B-rep edges to write: by default shared and free ones, not seams or degenerate edges), `extras` (our `userData`: source ids, properties, parts and `closed` where the source says whether a part's faces bound solids, scene material indices, face, edge, curve and point sources, and on the root the unit and up axis below it, millimetres and Z; default false); the root is a `Scene` holding the top objects, each turning millimetres and Z up into metres and Y up in its own matrix, as glTF's top nodes do, `keepEmpty` (keep tree nodes with nothing to draw beneath them; default false).
- **glTF** (`glb`, `gltfJson`, `gltfDocument`): the same, with `extras` for glTF's `extras`, plus `hidden` (`'omit'`, the default, or `'extension'` for `KHR_node_visibility`), `unlitLines` (`KHR_materials_unlit` on edges, curves and points), `exclude` (leave edges, curves or points out even when the view has them), `compression` (`'draco'`, the default, `'meshopt'` or `'none'`), `dracoPositionBits` (default 14), `instancing` (`EXT_mesh_gpu_instancing`; default false) and `keepEmpty`. A primitive with more faces than `MAX_DRACO_FACES` (65,535) with Draco and extras throws a `GltfError`. The encoders (WebAssembly) are imported when first used. `gltfJson(scene, view, 'model.bin', opts)` gives the JSON and the buffer it names; with `undefined` for the name, the buffer is embedded as base64.
- **3MF** (`threemf`): `name`, `defaultColor`, `colors` (`'basematerials'`, the default, or the Materials extension's `'colorgroup'`), `build` (`'parts'`, an item per placed part, or `'assembly'`), `nonManifold` (`'omit'` or `'surface'`: for meshes that aren't closed manifolds, and parts whose `closed` is false), `hidden` (`'omit'` or `'keep'`), `extras` (each item's names as `cadio:path` metadata; default false). The model is in the scene's source unit (`Scene.sourceUnit`: a STEP file's root product's, a 3MF's, glTF's metres) where 3MF has a name for it, else millimetres. Meshes are welded by exact position (3MF wants shared vertices), and a mesh that isn't a closed, consistently oriented manifold isn't written as a solid.

### 6.13 `cli` (`cad-io-cli`)

The two tools (see Command-Line Tools), also as functions: `step2json(argv, io)` and `stepconv(argv, io)` take the arguments as the command line has them and the streams to use (`{stdin, stdout, stderr}`), and return the exit code. The `bin/` scripts call them with the process's arguments and streams; tests and other programs can call them with their own.

```
import { stepconv, step2json } from 'cad-io-cli';

const io = { stdin: process.stdin, stdout: process.stdout, stderr: process.stderr };
const code = await stepconv(['assembly.stp', '-o', 'assembly.glb', '--with-edges', '-q'], io);   // 0
await step2json(['-m', 'semantic', 'assembly.stp', '-o', 'assembly.sem.json', '-q'], io);        // 0
```

## 7 Test Corpus

517 real-world Part 21 files, 421 MB, 4.9 M instances, listed in `test_corpus/files.txt` and fetched by `test_corpus/third_party/pull.sh` (not in the repository):

| Files | Source |
|---|---|
| 438 | the gkv311 OCCT dataset (https://github.com/gkv311/opencascade-dataset): OCCT's test data plus files from public OCCT bug reports, including the CAx-IF and ProSTEP test rallies' models from many CAD systems (`trj*`, `tr8_*`, `tr9_*`) and the bicycle assembly (`Median_cx-fs01_bicycle.stp`, 25 MB, 802 occurrences) |
| 60 | NIST's MBE PMI models (CTC, FTC, STC, D2MI; AP203 and AP242 editions 1-4) |
| 19 | the test files of `occt-import-js` |

By schema, as each file's `FILE_SCHEMA` names it:

| Application protocol | Schema names in the files | Files | Bound with |
|---|---|---|---|
| AP203 edition 1: Configuration controlled 3D design of mechanical parts and assemblies | `CONFIG_CONTROL_DESIGN`, alone or with CAx-IF modules (`GEOMETRIC_VALIDATION_PROPERTIES_MIM`, `SHAPE_APPEARANCE_LAYER_MIM`, `SHAPE_APPEARANCE_LAYERS_GROUPS`, `COLORS_AND_LAYERS`) | 62 | `ap203` |
| AP203 edition 1 as extended for the CAx-IF test rounds (by the names: colours and layers, validation properties, associative text) | `CCD_CLA_GVP_AST`, `CCD_CLA_GVP_AST_ASD`, `CCD_CLA_CVP_AST` | 13 | `ap242` (no dictionary) |
| AP203 edition 2: Configuration controlled 3D design of mechanical parts and assemblies | `AP203_CONFIGURATION_CONTROLLED_3D_DESIGN_OF_MECHANICAL_PARTS_AND_ASSEMBLIES_MIM_LF`, `CONFIGURATION_CONTROL_3D_DESIGN_ED2_MIM_LF` | 45 | `ap203e2` |
| AP214: Core data for automotive mechanical design processes | `AUTOMOTIVE_DESIGN`, and its conformance classes `AUTOMOTIVE_DESIGN_CC1`, `_CC2`, `_CC02`, `_CC04`, `_CC6` | 345 | `ap214e3` |
| AP242: Managed model-based 3D engineering | `AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF`: edition 1 (28 files), 2 (5), 3 (9) and 4 (4), by the object identifier | 46 | `ap242` |
| AP209 edition 1: Composite and metallic structural analysis and related design | `STRUCTURAL_ANALYSIS_DESIGN` | 4 | `ap242` (no dictionary) |
| AP227: Plant spatial configuration | `PLANT_SPATIAL_CONFIGURATION` | 1 | `ap242` (no dictionary) |

| Application protocol | Schema names in the files | Files | Bound with |
|---|---|---|---|
| A construction history test of the CAx-IF (parametric and procedural models) | `CONSTRUCTION_HISTORY_SCHEMA` | 1 | `ap242` (no dictionary) |
| | | 517 | |

A file whose schema has no dictionary is bound with AP242's, with a diagnostic: geometry, topology, products and presentation are common to all of these, and entities AP242 doesn't know are bound by position. The files include deliberately broken ones (unbalanced quotes, dangling and out-of-range references), files from writers with known quirks, wireframes, surface models, tessellated geometry, voids, external references and files over 50 MB. `test_corpus/subsets/` holds per-part subsets of 30 of them (188 parts), made by `corpus_checks/make_subsets.ts` (see Steps). See `test_corpus/README.md` for versions.

# 8 Tests

- **`npm test`**: the package tests, 437 of them (`node:test`, in each package's `test/`), on small inline fixtures and a few corpus files. Tests that need a built backend or the corpus skip when it's missing; with everything built, they take about 20 s.
- **`npm run typecheck`**: `tsc` over the packages (no output; the sources run as they are).
- **`cargo test`** in `packages/tess-truck/native`: 3 native tests of the truck wrapper (B-spline clamping, the volume sign guard).
- **Browser checks**, run by hand: `node packages/tess-workers/test/browser-check.ts` and `node packages/tess-cache/test/browser-check.ts` serve the sources and run the pool's and IndexedDB's scenarios in headless Chrome; `node packages/viewer/test/browser-check.ts` runs the viewer on models in each format (see Viewer).
- **Corpus checks** (`corpus_checks/`, 27 scripts, each with its last output committed as a `.log` beside it): every corpus file through each stage of the pipeline (parsing, writing, raw JSON, binding, semantic extraction, import, tessellation, export, and import back from each exported form), compared with OCCT (through `occt-import-js`) and with each output format's own reader or validator. Below.

The code was also checked by deliberately breaking it (a few dozen planted bugs for each piece of work, one at a time) and confirming the tests fail; the gaps found that way became tests.

## 8.1 Corpus Checks

They need the test corpus (and some the backends built or the subsets made; see Steps), and run from any directory. Those that read the whole corpus take an optional list file first (default `test_corpus/files.txt`; paths relative to `test_corpus/third_party/`), and the tessellating ones a maximum file size in MB second, with a low default so a first run is quick; the committed logs used the values in the commands below. The longer ones print progress on standard error. Their timings are examples from one machine; they vary widely with the machine and its power settings. `corpus_checks/README.md` has how to compare a run with its log.

### Parsing and writing

- `parser_corpus.ts` parses every file and decodes every instance, tallies the diagnostics and decode errors, and checks each instance's attributes against the schema dictionaries (slot counts, derived slots, unknown entities). About 12 s: `node corpus_checks/parser_corpus.ts`
- `parser_lenient_quotes.ts` lists the files and strings where the lenient end-of-string rule (a lone `'` ends a string only before `,` or `)`) reads differently from the strict one. About 4 s: `node corpus_checks/parser_lenient_quotes.ts`
- `parser_bench.ts` measures scan and decode speed and the index's memory for the files it's given (paths relative to `test_corpus/third_party/`). Seconds: `node corpus_checks/parser_bench.ts gkv311-opencascade-dataset/step/Median_cx-fs01_bicycle.stp`
- `writer_corpus.ts` parses every file, writes it, parses the output and compares every instance, the header and the DATA sections; then writes again (the bytes must be identical) and checks the fast path that copies instances' source bytes the same way. Expect 0 differences. About 1.5 min: `node corpus_checks/writer_corpus.ts`

- `json_corpus.ts` takes every file through raw JSON and back (parse, JSON, load, Part 21, parse), comparing every instance at each step; it also checks that `JSON.parse` accepts the JSON, that writing it again gives the same bytes, and that NDJSON loads back the same. Expect 0 differences. About 1 min: `node corpus_checks/json_corpus.ts`
- `number_kinds.ts` binds every integer and real to its schema slot and tallies what the schema says the slot holds (INTEGER, REAL, NUMBER, a SELECT, no context): context alone can't tell integers from reals, which is why raw JSON keeps the written form. About 1 min: `node corpus_checks/number_kinds.ts`

**Schemas and binding**

- `schema_corpus_check.ts`, the first dictionary check, with a crude tokenizer written before the parser existed; superseded by `parser_corpus.ts`, kept because `schema_xdict.ts` reads its log. About 7 s: `node corpus_checks/schema_corpus_check.ts > corpus_checks/schema_corpus_check.log`
- `schema_xdict.ts` compares the four dictionaries' attribute layouts with AP242's, and looks up in AP242 the entities that log reports as unknown to a file's own schema. Under a second: `node corpus_checks/schema_xdict.ts corpus_checks/schema_corpus_check.log`
- `bind_corpus.ts` binds every instance with value checks and tallies how instances bind and every diagnostic; it cross-checks each file's reverse references against its forward ones, and `instancesOf` against `isInstanceOf`. About 20 s: `node corpus_checks/bind_corpus.ts`

**Semantics and import**

- `semantic_corpus.ts` runs every semantic extractor on every file and tallies products, occurrences (and whether each got a placement), styles, layers, validation properties and diagnostics. About 5 s: `node corpus_checks/semantic_corpus.ts`
- `products_occt.ts` checks product structure and placements against OCCT's assembly reader (`occt-import-js`): every B-rep vertex of every product instance, placed with our transforms, must be among OCCT's mesh vertices. Many minutes, since OCCT tessellates every file: `node corpus_checks/products_occt.ts test_corpus/files.txt 30`
- `validation_occt.ts` compares the volume, area and centroid validation properties we extract with those computed from OCCT's meshes, for files with one root. Minutes: `node corpus_checks/validation_occt.ts test_corpus/files.txt 5`
- `subset_corpus.ts` builds a subset for every shape representation the product structure reaches, as the importer does, and checks that it parses cleanly, matches its source instance for instance, and lacks only references dangling in the source. About 10 s: `node corpus_checks/subset_corpus.ts`
- `import_corpus.ts` imports every file into a scene and checks it (`validate`, agreement with the semantic layer, every part's payload parsed), tallying parts, materials and layers; a second argument `none` skips the payload checks. About 10 s: `node corpus_checks/import_corpus.ts`
- `dangling_refs.ts` lists the importer's warnings of dangling references and the scan's references beyond the largest instance id, to compare with OCCT's "Unresolved Reference" messages. About 16 s: `node corpus_checks/dangling_refs.ts`

**Tessellation**

- `scene_occt.ts` checks the whole path (import, our OCCT backend, placement) against `occt-import-js`'s import of the same file: sampled vertices must lie on OCCT's surfaces, and sampled triangles must have OCCT's colours. About 4 min: `node corpus_checks/scene_occt.ts test_corpus/files.txt 5`
- `truck_occt.ts` runs every brep part through both backends (each in a worker pool, so hangs time out) and compares them part by part and face by face: truck's failures, missing faces, edge kinds, edge points off the mesh, area, faces facing the other way, faces contradicting their loops, its workarounds; and both backends' surface area against the file's validation properties. `DUMP=<dir>` writes each problem part's subset; `WORKERS=<n>` sets each pool's size (default 4). About 16 min: `node corpus_checks/truck_occt.ts test_corpus/files.txt 30`
- `backend_speed.ts` times the two backends against each other as `stepconv` runs them: every file's brep parts through each backend's worker pool, back to back, with the defaults; it prints each backend's outcomes and run time, and, over the parts both mesh, the time and triangles of each by part size, then the slowest parts. About 25 min (truck's

timeouts take most of it): `node --max-old-space-size=8192 corpus_checks/backend_speed.ts test_corpus/files.txt 1000`

- `truck_bench.ts` times truck module variants (the `.wasm` files it's given; default the built one) and our OCCT backend, single-threaded, over the subsets. About 1 min per variant, with the CPU otherwise idle: `node corpus_checks/truck_bench.ts`
- `cache_check.ts` checks `tess-cache` with each backend over the subsets: every result read back from a new file store, and from a memory store, equals the backend's own; it times the backend alone and a first and second cached run, and checks that a timeout is cached and answered from the store. About 2 min: `node corpus_checks/cache_check.ts`
- `weld_survey.ts` welds every part's mesh by position (exactly, and within two tolerances), drops degenerate triangles, and classifies each mesh as closed, open, misoriented or non-manifold (3MF's rules for solids), split by whether the B-rep has free edges. About 4.5 min: `node --max-old-space-size=8192 corpus_checks/weld_survey.ts test_corpus/files.txt 1000`

**Export**

- `export_view.ts` checks the export view against `partInstances` for every file: each drawable part instance once at its path, world transforms with the mesh's origin, visibility, default materials, every triangle with the material the scene resolves for its face, equal instances sharing a look. About 3 min: `node corpus_checks/export_view.ts test_corpus/files.txt 1000`
- `roundtrip_corpus.ts` exports every file with extras as three.js JSON, as GLB (uncompressed, Draco, meshopt, instanced) and as 3MF (an item per placed part, and components), reads each back with `threejs-import`, `gltf-import` and `3mf-import`, and compares it with what was written (`viewDifferences`: placements, names, visibility, every triangle's world corners, face id and colour, every edge, curve and point; lossless forms to 1e-4 mm, the codecs within their step, instancing within float32 rounding, 3MF without lines and within its float32 text); `PLANT=` perturbs the imported scenes to show it fails. About 31 min: `node --max-old-space-size=12000 corpus_checks/roundtrip_corpus.ts test_corpus/files.txt 1000`
- `threejs_corpus.ts` exports every file to three.js JSON with edges, curves and extras, loads it with `THREE.ObjectLoader` and compares it with the export view: objects in tree order, world matrices, index counts, every triangle's colour and opacity. About 3.5 min: `node --max-old-space-size=8192 corpus_checks/threejs_corpus.ts test_corpus/files.txt 1000`
- `gltf_corpus.ts` exports every file as GLB with edges, curves, hidden instances and extras, uncompressed, validates it with the Khronos glTF-Validator (errors are problems), reads it back with glTF-Transform and compares it with the export view: a node per placed part in tree order, visibility, world matrices, triangles and colours, one mesh per look. It also writes each file with Draco and with meshopt and compares the decoded triangles with the view, face by face (Draco's by `_FACE`, meshopt's by range) within what quantization allows, and every line and point vertex; and with instancing, every placement as a tree node or an instance at its world matrix, and no empty nodes. About 4 min: `node --max-old-space-size=8192 corpus_checks/gltf_corpus.ts test_corpus/files.txt 1000`
- `threemf_corpus.ts` exports every file as 3MF (open meshes as surfaces, hidden instances kept), reads it back with lib3mf in strict mode and compares it with the export view: an item per placed part at its world transform, each object's type against lib3mf's manifold check and its volume's sign, every triangle's corners and colour; and the assembly form composed to the same objects and transforms. About 5.5 min: `node --max-old-space-size=8192 corpus_checks/threemf_corpus.ts test_corpus/files.txt 1000`
- `stepconv_corpus.ts` runs `stepconv` with its defaults on every file in each format, in process, and spawns the command for a sample (third argument, default 20 files), reading each output back with its format's tools (glTF through the Draco decoder) and comparing it with the export view (part names and triangle counts in tree order, and no extras); the command's outputs must be byte-identical to the in-process ones. About 20 min: `node --max-old-space-size=8192 corpus_checks/stepconv_corpus.ts test_corpus/files.txt 1000`

# 9 Roadmap

Done in 0.1.0: the STEP parser and writer, schema binding and semantic extraction, `step2json`; the scene model and STEP importer; the OCCT and truck backends with workers and caching; the three.js, glTF and 3MF exporters and

stepconv. Done in 0.2.0: the viewer; levels of detail in the scene model; importers of glTF/GLB, three.js JSON and 3MF, through which the viewer reads every format; AP242 tessellated geometry read as it is; B-rep face measures; leaf products as placed parts, so round trips keep the tree; exports in the input's frame.

Next, roughly in order:

1. **Publishing to npm:** emit JavaScript (`rewriteRelativeImportExtensions` is already on), package names and scope, per-package READMEs and licenses, the prebuilt WebAssembly modules in their packages.
2. **Large files in the browser:** reading instance bytes on demand instead of holding the file, 64-bit offsets.
3. **More of STEP:** PMI (semantic and graphical), user-defined attributes, materials, SHUOs, per-curve colours, named coordinate systems.
4. **More formats** (the scene and tessellator interface were designed for them), by value to a viewer:
   - **JT** (ISO 14306; Siemens Teamcenter): a pure-TypeScript importer of its meshes (levels of detail, so no tessellation needed), scene graph and properties; v9 and v10 compression both needed; specifications in `third_party/jt/`, free to implement against (the terms are summarized in `CLAUDE.md`);
   - **IGES**: a small parser plus the OCCT backend (`format: 'iges'`, subsets with renumbered pointers);
   - **OCCT BREP** (`.brep`), OCCT documents (`.xbf`) and **FreeCAD** (`.FCStd`, whose `.brp` shapes OCCT reads);
   - exporting STEP (the writer exists) and JT.

Not planned for now: AP242 XML (`.stpx`).

# 10 Backlog

Known work not yet scheduled; the detail behind each is in the topic notes in `docs/claude/`.

**Parser and extraction**

- PMI, semantic and graphical (`rec_pracs_pmi_v41.pdf` in `third_party/mbx-if-recpractices/`); 2D drawings (`GEOMETRICALLY_BOUNDED_2D_WIREFRAME`, skipped today) go with it.
- User-defined attributes; materials and densities; kinematics; SHUOs (specified higher-usage occurrences: a style, visibility or replacement that applies to one instance deep in an assembly, named by its path of occurrences; the importer doesn't read them yet, though it does apply per-instance styles given in the context of a chain of occurrences).
- AP242 tessellated geometry beyond the CAx-IF practice: curved triangles (`CUBIC_BEZIER_TRIANGULATED_FACE`), tessellated faces in B-rep shells (`TESSELLATED_FACE_SURFACE`), per-vertex and per-triangle colours (`STYLED_TESSELLATED_ITEM_WITH_COLOURS`): left to the backend, or ignored; tessellated validation properties not checked.
- Per-instance style resolution beyond what the importer does; per-curve colours in mixed parts.
- Named coordinate systems: placements among a representation's items that nothing uses (shown, not applied).
- Part 21 edition 3 constant references and anchors (sections skipped with a warning today; no corpus file has any).
- Files over 4 GiB (64-bit offsets); on-demand range reading; streaming raw conversion.
- More dictionaries (AP209, AP214 ed1); WHERE rules and complex-instance validity not checked.

**Tessellation**

- An adaptive meshing mode: the deflections driven by each part's size (its bounding box's diagonal, say) rather than absolute (absolute 0.1 mm gives a 32 m torus 3.9 M triangles), perhaps as a fourth profile beside coarse, medium and fine.
- A maximum chord (edge) length, as another CAD system's meshing profiles have (10 m in each); neither backend has one.
- OCCT at tighter angles: `bug30790.step`'s parts #11-21 and #24 take 0.5 s each at 28.6 degrees, 24-44 s at 15 (medium) and time out at 12.5 and below, whatever the distance; find out why (the same file has the corpus's most unmeshed faces).
- OCCT: ShapeFix's split of a face whose hole touches its seam (bicycle #23723, left as is); Delabella for faces with thousands of holes (6 times as fast on `bug24024_slow_import.stp`); size cuts (CJK code-page tables, writer code).
- truck: wireframe curves (`TRIMMED_CURVE`, `COMPOSITE_CURVE`); geometric sets' surfaces; cones, surfaces of revolution and rational B-splines it meshes badly (no workaround short of patching truck).

**Import**

- Levels of detail in use: the scene holds several per part, but the viewer and the exporters use one at a time (`chooseLod`); no level is chosen by distance yet.

**Export**

- The 3MF Production extension.
- Exporting STL and OBJ (the mesh helpers are there).

**Viewer**

- An option to switch ambient occlusion from N8AO to three.js's `GTAOPass` (no dependency), to compare.
- Optional `three-mesh-bvh` for faster picking on large meshes (more complexity; less reliable on some mobile GPUs).
- Several files open at once, each a top node of the tree named after its file (the viewer opens one model at a time).

**Open questions**

- Which AP242 edition stepcode's schema is (later than edition 1; it binds files of editions 1-4 alike).
- The ISO EXPRESS schemas' redistribution terms, for the generated dictionaries (names and structure only).
- JT's "combining formats" clause, as it applies to converting JT to glTF or 3MF.

# 11 Contributing

Pull requests are welcome; see `CONTRIBUTING.md`. Their authors sign the Contributor License Agreement once, by a comment on their first pull request, which a GitHub Action (`.github/workflows/cla.yml`) asks for and records.

# 12 License

The GNU Affero General Public License, version 3 only (AGPL-3.0-only, `LICENSE.txt`, since 0.2.0: versions and commits prior to 0.2.0 were Apache-2.0 except where noted, and remain available under it), except `packages/tess-occt`, which is LGPL-2.1 with the Open CASCADE exception, as OCCT is (`packages/tess-occt/LICENSE.txt`), and the demo models in `test_corpus/demo/`, which are Marsette Vona's under CC BY-SA 4.0 (`test_corpus/demo/LICENSE.txt`; deployed with their license beside them, `content/demo/LICENSE.txt`). The other code we build against keeps its own license: OCCT LGPL-2.1, stepcode BSD-3-Clause, truck Apache-2.0, glTF-Transform and fflate MIT. The code here is written independently, using OCCT as a reference and test oracle only, not translated from it.